Программа экзамена

Advanced Machine Learning (girafe-ai, МФТИ) Осень 2025

Примечание. Пункты, помеченные *, и все их подпункты являются продвинутыми и необязательными.

Глава 1. Введение и базовая методология

1. Определение и методология машинного обучения (ML)

a) На каких дисциплинах основано
b) Разница между классическим программированием и ML
c) Базовые термины ML

2. Приложения ML (подобласти)

a) Назвать минимум 3 и привести практические примеры

3. Датасеты

a) Способы находить датасеты (минимум 3 источника)

4. Разведочный анализ данных (EDA)

a) Зачем нужен EDA для моделирования, примеры
b) Типичные шаги EDA

Глава 2. Вероятностные основы и постановка задач

5. Оценка максимального правдоподобия (MLE)

a) Определение правдоподобия
b) Следствия свойства i.i.d.
c) Формула отрицательного лог-правдоподобия

6. Таксономия задач ML

a) Обучение с учителем и без учителя, примеры
b) Назвать другие категории
c) Практические примеры (минимум 3 для каждого типа)
d) Постановка практической задачи, основные сущности

7. Наивный байесовский классификатор

a) Формула Байеса
b) Основные предположения
c) Случай непрерывных признаков

Глава 3. Библиотека sklearn

8. Библиотека sklearn

a) Базовые классы, их функции и использование

b) Важность пайплайнов, пример

Глава 4. Линейная регрессия, оптимизация и регуляризация

9. Регрессионная модель

a) Предположения

b) * Почему это важно

10. Линейная регрессия

a) Мотивация функции потерь MSE

i) доказательство для гауссовского случая

b) Аналитическое решение для весов (матричная форма)

i) доказательство

c) Теорема Гаусса–Маркова, BLUE

i) * доказательство

11. Градиентный спуск для линейной регрессии

a) Почему не используют матричное решение

b) Стохастический градиентный спуск, преимущества

12. Регуляризация в линейных моделях

a) Мотивация регуляризации

b) Гребневая регрессия Ridge (L2)

i) Формула решения

c) Lasso-регуляризация (L1)

d) Вероятностная интерпретация

e) Elastic Net

f) Свойства регуляризаций

13. Метрики регрессии MSE, MAE, RMSE, MAPE, SMAPE, R2

a) Когда их используют

14. Параметры и гиперпараметры

a) Определения, свойства, использование, примеры

Глава 5. Валидация и оценка качества

15. Валидация и оценка модели

a) Недообучение и переобучение, определения и способы обнаружения

b) Способы разбиения датасета

i) Что такое хорошее разбиение, идеальный и практический варианты
ii) 4 основных типа разбиений

c) Этапы построения модели
i) Train, validation, test, мотивация каждого
ii) Кросс-валидация
iii) Как выбрать организацию этапов

d) Утечки данных

Глава 6. Линейная классификация, метрики и SVM

16. Линейная классификация

a) Постановка задачи классификации
b) Варианты типов меток
c) Определение линейного классификатора и геометрическая интерпретация
d) Маржа, определение и ключевое свойство
e) Выбор весов
i) Почему для классификации много разных функций потерь

17. Логистическая регрессия

a) Базовое предположение и геометрическая интерпретация
b) Сигмоида как предсказание вероятности, свойства
c) Формула логистической функции потерь
i) вывод через MLE
d) Свойство калибровки вероятностей
e) Обобщённая линейная модель

18. Стратегии многоклассовой классификации

a) One-vs-one, one-vs-all и их свойства

19. Метрики классификации

a) Определение метрики в ML
i) Свойства, сравнение с функцией потерь
b) Матрица ошибок (бинарная, многоклассовая)
i) Типы ошибок
ii) Правило присвоения меток
c) Accuracy, balanced accuracy, precision, recall, F1-score
i) Sensitivity, specificity и другие
ii) Несбалансированная стоимость ошибок, примеры, как исправлять дисбаланс
d) ROC AUC
i) Свойства ROC и ROC-AUC
ii) PR AUC
e) Многоклассовые расширения
i) Микро- и макро-усреднение

20. Матричные нормы

a) p-нормы, самые важные частные случаи

21. Метод опорных векторов (SVM)

a) Геометрическая интерпретация
b) Оптимизационная задача для линейно разделимого случая
    i) * Двойственная задача
c) Неразделимый случай, slack-переменные
d) Hinge loss
e) Ядровой трюк, свойства ядер, примеры ядер

Глава 7. Методы без учителя и методы ближайших соседей

22. Метод главных компонент (PCA)

a) Связь с SVD
b) Теорема Эккарта–Юнга, доказательство
    i) способ через собственное разложение или через подстановку
c) Две альтернативные итеративные формулировки
d) Предобработка для PCA

23. k ближайших соседей (kNN) для классификации и регрессии

a) Параметры и гиперпараметры

24. Индексы для kNN

a) Зачем нужны индексы
b) Идеи индексов для kNN от простых к сложным
c) k-dimensional tree (kd-tree)
d) Annoy
e) HNSW
    i) Построение графа
    ii) Использование графа
f) Примеры векторных баз данных

25. The bitter lesson (Горький урок)

a) Основные выводы статьи

Глава 8. Деревья решений, ансамбли, интерпретируемость

26. Деревья решений

a) Определение жадных алгоритмов
b) Процедура построения
c) Информационные критерии, энтропия, индекс Джини
d) Ответ в листе

e) Обработка пропусков
f) Бинаризация
g) Обрезка (pruning), варианты C4.5 и CART
h) Гиперпараметры (минимум 4 основных)

27. Разложение bias–variance–noise

a) Случайность в ошибке
    i) Интуиция каждой компоненты
b) Доказательство для MSE
c) Компромисс bias–variance
    i) Анализ основных моделей
d) За пределами MSE
e) Double descent

28. Бэггинг

a) Мотивация через bias–variance
b) Бутстрэп
c) Метод случайных подпространств
d) Random Forest

29. Задача обнаружения аномалий

a) Isolation Forest
b) One-Class SVM

30. Градиентный бустинг (GB)

a) Процедура построения для MSE
b) Бустинг как градиентный спуск
    i) вывод для кросс-энтропии
c) Выбор базовой модели
d) Параллелизм

31. Библиотеки градиентного бустинга

a) XGBoost, LightGBM, CatBoost
b) Плюсы и минусы каждой

32. Интерпретируемое ML

a) Таксономия по свойствам
b) Линейные модели, важности в деревьях
c) LIME
d) Подход SHAP (Shapley additive explanations)
    i) Аксиомы, формула
    ii) SHAP-значения для ML
    iii) KernelSHAP, TreeSHAP
e) GradCAM

33. Оптимизация гиперпараметров (HP)

a) Классы задач оптимизации
b) Основные подходы к оптимизации HP
c) Библиотеки для оптимизации HP

Глава 9. Нейросети, матричные производные и базовые слои

34. Матричное исчисление и производные матричных операций

a) Производная скалярного произведения
   i) Для двух векторов
   ii) Для матрицы и вектора

35. Концепция нейросети

a) Мотивация нейросетей
b) Задача XOR
c) Определения слоёв
   i) Графическая нотация
d) Определение эмбеддинга

36. Линейный слой (Linear layer)

a) Все названия линейного слоя
b) Формула
c) Гиперпараметры, число параметров
d) Инициализация
e) Теорема универсальной аппроксимации

37. Функции активации

a) Примеры (минимум 4)
b) Основные свойства (сравнительный анализ)

38. Обратное распространение ошибки (backpropagation)

a) Правило цепочки
b) Посчитать простой пример вручную

39. Многоклассовая классификация в нейросетях

a) Softmax
   i) Основные свойства
   ii) Численная стабильность
b) Потеря кросс-энтропии
   i) Определения информационного содержания и энтропии
   ii) Мотивация и смысл кросс-энтропии
   iii) Perplexity
   iv) Дивергенция Кульбака–Лейблера

Глава 10. Аппаратное обеспечение и PyTorch

40. Компьютерное железо для ML

a) Архитектура фон Неймана

i) Принцип локальности данных

ii) Поток данных при обучении модели

b) Таксономия Флинна

i) Место CPU и GPU

ii) CPU vs GPU, основные различия

c) NVIDIA CUDA, что это

41. Библиотека PyTorch

a) Зачем нужна ещё одна матричная библиотека

b) Доступные бекенды

c) Архитектура, основные модули и классы

i) nn.Module, Sequential, Optimizers, Schedulers, Criterions

d) Dataflow в PyTorch, классы и их назначение

Глава 11. Обработка текста и векторизация

42. Обработка текстов

a) Подходы к токенизации (минимум 3)

b) Лемматизация, стемминг

i) Назвать базовые стеммеры

43. Векторизация слов

a) One-hot encoding

i) Недостатки

b) Word2Vec, GloVe, FastText

i) Постановка задачи, функция потерь, признаки

44. Векторизация текста

a) Bag-of-words

b) TF-IDF

c) * BM25

Глава 12. Сглаживание, индуктивные смещения и оптимизация

45. Экспоненциальное сглаживание

a) Формула

b) Мотивация названия

46. Индуктивные смещения (inductive bias) в Deep Learning

a) Регуляризация
b) Batch normalization, layer normalization
c) Dropout
    i) Отличия режимов обучения и инференса
d) Аугментации
    i) Основные библиотеки
    ii) * Test-time augmentations (TTA)

47. Методы оптимизации

a) Gradient descent, SGD, Momentum, RMSProp, Adam
b) Muon
c) Расход памяти для каждого

Глава 13. Ранжирование и uplift modelling

48. Ранжирование

a) Постановка задачи
    i) Отличия от регрессии и классификации
b) Оффлайн-метрики
    i) mAP
    ii) NDCG
    iii) MRR
c) Онлайн-метрики
    i) Как измеряются
    ii) Дерево метрик, North star метрика
d) Функция потерь RankNet
    i) Что она предсказывает
    ii) * Lambda-gradients
    iii) Другие функции потерь (просто назвать)

49. Uplift modelling

a) Постановка задачи, отличие look-alike и response моделей
b) Two-model uplift technique
c) Class transformation technique
d) Библиотеки для uplift modelling
e) Примеры применения

1 Определение и методология машинного обучения (ML)

Машинное обучение (Machine Learning, ML) — это подход к построению моделей и алгоритмов, которые улучшают качество решения задачи за счёт *опыта*, получаемого из данных, а не за счёт явного программирования всех правил обработки входов. Важнейшая особенность ML в сравнении с «обычным кодом» — **статистическая природа моделей**: данные рассматриваются как реализация случайных величин, а качество описывается вероятностно-статистическими критериями.

В типичной постановке рассматривается датасет из наблюдений (объектов), для которых задан набор признаков и (в задачах обучения с учителем) известен целевой ответ. Обучение модели сводится к подбору параметров так, чтобы минимизировать ошибку предсказаний на данных. На практике это делается через оптимизацию функции потерь или эквивалентную ей постановку (например, через максимизацию правдоподобия):

$$\hat{\theta} = \arg\max_{\theta} p(\mathcal{D} \mid \theta), \qquad \hat{\theta} = \arg\max_{\theta} \log p(\mathcal{D} \mid \theta), \qquad \hat{\theta} = \arg\min_{\theta} \big(-\log p(\mathcal{D} \mid \theta)\big).$$

При предположении независимости наблюдений (i.i.d.) правдоподобие раскладывается в произведение по объектам, а лог-правдоподобие — в сумму:

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{n} p(x_i, y_i \mid \theta), \qquad \log p(\mathcal{D} \mid \theta) = \sum_{i=1}^{n} \log p(x_i, y_i \mid \theta).$$

Методологически ML-процесс можно представить как последовательность связанных шагов:

- формализация задачи (что предсказываем/оптимизируем: регрессия, классификация, кластеризация и т.д.);
- формирование датасета и выбор представления данных (матрица признаков, вектор/матрица таргетов);
- выбор семейства моделей и гипотез (например, независимость признаков в Naïve Bayes или форма линейной зависимости в линейных моделях);
- выбор функции потерь (и/или вероятностной модели), обучение параметров путём оптимизации;
- оценка качества и проверка *обобщающей* способности на отложенных данных (валидация/тест с контролем переобучения).

1.1 На каких дисциплинах основано

Машинное обучение является междисциплинарной областью и опирается на следующие базовые направления:
Теория вероятностей и математическая статистика. Данные рассматриваются как выборки из распределения; используются понятия случайных величин, распределений, условных вероятностей, ожиданий/дисперсий, а также методы оценивания параметров (например, метод максимального правдоподобия). Предположение i.i.d. часто используется как базовое для вывода формул и корректной оценки качества.
Линейная алгебра. Данные обычно представляют в виде матрицы признаков (design matrix), параметры модели — в виде векторов/матриц, а многие модели являются композициями линейных отображений.
Математическая оптимизация. Обучение параметров — это оптимизационная задача (минимизация потерь или максимизация правдоподобия), где применяются градиентные и/или нетривиальные методы.
Теория алгоритмов и вычислительная математика. Практическая применимость моделей определяется вычислительной сложностью, устойчивостью численных методов и ограничениями вычислительных ресурсов.

1.2 Разница между классическим программированием и ML

Различие между классическим программированием и ML удобно описывать через то, **что является «ядром» решения**.
Классическое программирование: правила задаются явно программистом.
Данные (Input) + Правила (Program) ⟶ Ответ (Output).
Логика обработки фиксирована и изменяется только при модификации кода.

Машинное обучение: правила не задаются вручную, а **извлекаются из данных** в процессе обучения.
Данные (Input) + Правильные ответы (Targets) ⟶ Модель (Model),
Данные (Input) + Модель (Model) ⟶ Предсказание (Prediction).
Иными словами, в ML программист проектирует *процедуру обучения* и выбирает семейство моделей, а конкретное «правило» задаётся найденными параметрами модели.
Дополнительно важно, что ML-решения по своей природе статистические: качество оценивается метриками/ошибками на выборке, а корректность определяется обобщением на новых данных и контролем переобучения.

1.3 Базовые термины ML

Ниже приведён базовый тезаурус, используемый в курсе.

- **Датасет (dataset)** — совокупность данных для обучения/оценки. В обучении с учителем часто используется представление

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n},$$

где x_i — признаки объекта, y_i — целевой ответ.
- **Наблюдение / объект / datum (observation)** — один элемент выборки (одна строка в таблице данных). Часто предполагается, что наблюдения i.i.d. (независимы и одинаково распределены).
- **Признак (feature)** — измеряемая характеристика объекта; в табличном виде соответствует столбцу.
- **Матрица признаков / design matrix** — матричное представление всех объектов и их признаков:

$$X \in \mathbb{R}^{n \times d},$$

где n — число объектов, d — число признаков.
- **Целевая переменная / таргет (target)** — то, что требуется предсказывать:

$$y \in \mathbb{R}^n \quad \text{(регрессия)} \qquad \text{или} \qquad y \in \{1, \dots, K\}^n \quad \text{(классификация)}.$$

- **Модель (model)** — параметризованное семейство функций/распределений, задающее правило получения предсказания из признаков, например

$$\hat{y}_i = f(x_i; \theta).$$

- **Предсказание (prediction)** — результат применения модели к входным данным: $\hat{y} = f(X; \theta)$.
- **Функция потерь (loss)** — числовая мера ошибки предсказаний, оптимизируемая при обучении. Примеры для регрессии:

$$\mathrm{MSE}(y, \hat{y}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \mathrm{MAE}(y, \hat{y}) = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|.$$

- **Параметры (parameters)** — величины, которые *обучаются* на данных (например, веса линейной модели).
- **Гиперпараметры (hyperparameters)** — настройки, задаваемые *до* обучения (например, коэффициент регуляризации, глубина дерева и т.п.).
- **Типы задач ML (таксономия)** — наиболее употребимые классы постановок: *supervised* (с учителем), *unsupervised* (без учителя), *semi-/self-supervised*, *reinforcement* и др.

2 Приложения ML (подобласти)

2.1 Назвать минимум 3 и привести практические примеры

В рамках прикладного машинного обучения принято выделять несколько ключевых подобластей, которые отличаются типом данных, постановкой задачи и используемыми моделями/архитектурами. Ниже приведены примеры таких подобластей и типовые прикладные кейсы.

1. **Обработка естественного языка (NLP, Natural Language Processing).**
Тип данных: тексты, речь (в дискретном или векторном представлении).
Типовые задачи:
   - машинный перевод;
   - классификация текстов (например, тематика, спам/не спам);
   - генерация текста (диалоговые системы, автодополнение).

   Практические примеры:
   - автоматическая модерация контента и фильтрация спама в сообщениях/почте;
   - чат-бот поддержки (классификация намерений + генерация/выбор ответа);
   - машинный перевод для многоязычного интерфейса продукта.
2. **Компьютерное зрение (Computer Vision).**
Тип данных: изображения/видео (пиксельные тензоры, признаки).
Типовые задачи:
   - детекция объектов (поиск объектов и их локализация);
   - семантическая сегментация (разметка классов по пикселям);
   - генеративные модели (синтез/восстановление изображений).

   Практические примеры:
   - контроль качества на производстве: детекция дефектов на конвейере по изображению;
   - медицинская визуализация: сегментация органов/новообразований на снимках;
   - видеоаналитика безопасности: обнаружение людей/техники в запрещённых зонах.
3. **Обучение с подкреплением (RL, Reinforcement Learning).**
Тип данных: взаимодействие агента со средой (состояния, действия, награды).
Типовые задачи: оптимизация стратегии поведения, управление в условиях неопределённости: последовательное принятие решений.
Практические примеры:
   - игровые системы, обучающиеся через самообучение и множество эпизодов (классически демонстрации возможностей RL);
   - робототехника и управление: обучение политик управления манипуляторами/дронами в симуляции с переносом в реальную среду;
   - оптимизация управления ресурсами (например, распределение нагрузки/очередей) как задача последовательного контроля.
4. **Машинное обучение на графах (Graph ML).**
Тип данных: графы (вершины, рёбра, признаки), сети взаимодействий.
Типовые задачи: выявление сообществ, предсказание связей, ранжирование/рекомендации на основе графовой структуры.
Практические примеры:
   - рекомендательные системы: рекомендации контента/товаров на основе графа пользователей и объектов;
   - антифрод: поиск аномальных паттернов в графе транзакций и связей;
   - анализ социальных сетей: обнаружение сообществ и инфлюенсеров.
5. **Ранжирование и рекомендательные системы (Learning-to-Rank, RecSys).**
Тип данных: события взаимодействия пользователь–объект, признаки пользователя/объекта.
Типовые задачи: упорядочить объекты по релевантности (поиск, ленты, рекомендации).
Практические примеры:
   - ранжирование документов в поисковой выдаче по запросу пользователя;
   - ранжирование карточек товаров/видео/новостей в персонализированной ленте;
   - рекомендательные системы, оптимизирующие метрики качества ранжирования (напри NDCG, MAP и др.).

3 Датасеты

3.1 Способы находить датасеты (минимум 3 источника)

В курсе выделяются несколько практических источников и подходов к поиску датасетов (как для учебных задач, так и для прикладных проектов):

1. **Google Dataset Search.** Специализированный поисковик по датасетам: позволяет находить наборы данных по тематике, смотреть метаданные и переходить к первоисточникам. :contentReference[oaicite:0]{index=0}
2. **Kaggle Datasets.** Крупнейшая платформа с датасетами и соревнованиями: удобно для быстрого старта, бенчмарков и воспроизводимых baseline-решений. :contentReference[oaicite:1]{index=1}
3. **Hugging Face Datasets.** Каталог датасетов (особенно популярен в NLP и смежных областях): предоставляет единый интерфейс загрузки и работы со многими наборами данных. :contentReference[oaicite:2]{index=2}
4. **UCI Machine Learning Repository.** Исторический репозиторий датасетов, широко используемый в обучении и для классических ML-бенчмарков. :contentReference[oaicite:3]{index=3}
5. **Papers with Code (каталоги датасетов и бенчмарков).** Удобен тем, что связывает датасеты с постановками задач, статьями и результатами моделей (как ориентир для выбора датасета под конкретную задачу). :contentReference[oaicite:4]{index=4}
6. **ML-соревнования и каталоги соревнований.** Источник прикладных датасетов и «живых» постановок задач, часто с чёткими метриками и протоколом оценки. :contentReference[oaicite:5]{index=5}

4 Разведочный анализ данных (EDA)

4.1 Зачем нужен EDA для моделирования, примеры

Разведочный анализ данных (Exploratory Data Analysis, EDA) — это этап предварительного исследования датасета до построения модели. Его цель — понять структуру данных, качество, распределения признаков и целевой переменной, выявить проблемы (пропуски, выбросы, ошибки утечки), а также сформировать корректные гипотезы о данных и ограничениях будущего моделирования. В материалах курса подчёркивается, что перед обучением любой модели важно проанализировать имеющиеся данные, поскольку поведение модели и итоговое качество напрямую зависят от структуры датасета. :contentReference[oaicite:0]{index=0}
Зачем EDA нужен для моделирования:

- **Проверка корректности данных и качества датасета.** EDA помогает находить пропуски, дубликаты, аномальные значения, ошибки ввода, несогласованность типов, некорректные единицы измерения. Такие дефекты напрямую ухудшают обучение и могут приводить к некорректным выводам.
- **Понимание распределений и масштаба признаков/таргета.** По распределениям можно оценить асимметрию, тяжёлые хвосты, редкие категории, а также понять, требуется ли преобразование (например, лог-преобразование) или робастные метрики/потери.
- **Выявление выбросов и их природы.** Выбросы могут быть как ошибками измерения, так и редкими, но важными режимами работы системы. Решение (удалять/каппировать/оставить) влияет на устойчивость модели.
- **Оценка связей между признаками и целевой переменной.** Корреляции, зависимость таргета от ключевых признаков и наличие нелинейностей помогают выбирать модельный класс (линейные модели vs деревья vs нейросети) и формировать направления feature engineering.
- **Предотвращение методологических ошибок.** EDA может выявить *утечки таргета* (target leakage), неверную схему разбиения на train/test, временную зависимость, нарушение i.i.d. и т.п.

Примеры, показывающие необходимость EDA:

- **Квартет Аскомба.** Четыре набора данных могут иметь практически одинаковые простые описательные статистики (среднее, дисперсии, корреляция), но при визуализации оказываются принципиально разными по структуре и форме зависимостей. Это демонстрирует, что без визуализации и EDA «сводные числа» могут вводить в заблуждение. :contentReference[oaicite:1]{index=1}
- **Пропуски и категориальные признаки.** Если в таблице есть категориальные признаки (например, фильтр, цвет глаз) и пропуски в них, то без EDA и корректной обработки (кодирование категорий, работа с NA) модель может либо упасть на этапе обучения, либо дать смещённые результаты из-за неправильного представления таких признаков. :contentReference[oaicite:2]{index=2}

4.2 Типичные шаги EDA

Ниже приведён типовой набор шагов EDA, используемый как практический чек-лист перед моделированием. Он соответствует общей логике EDA из материалов курса: сначала понять состав и качество данных, затем изучить распределения и связи, затем сформировать выводы для подготовки данных и выбора модели. :contentReference[oaicite:3]{index=3}

1. **Определение цели и единицы наблюдения.**
Зафиксировать постановку (что предсказываем и для какого объекта/момента времени), определить гранулярность данных (строка = объект/событие/период) и убедиться, что это согласовано с целью моделирования.
2. **Первичный обзор структуры датасета.**
Проверить размерность, список признаков, типы данных, доли пропусков, уникальность идентификаторов, наличие дубликатов строк/ключей.

5 Оценка максимального правдоподобия (MLE)

5.1 Определение правдоподобия

Пусть дан датасет наблюдений

$$\mathcal{D} = \{z_i\}_{i=1}^{n}, \qquad z_i = (x_i, y_i),$$

который считается сгенерированным некоторым вероятностным законом (распределением) с параметром θ.
Правдоподобие (likelihood) параметра θ — это вероятность (или плотность вероятности) получить именно наблюдённые данные $\mathcal{D}$ при условии, что истинный параметр равен θ:

$$\mathcal{L}(\theta \mid \mathcal{D}) = p(\mathcal{D} \mid \theta).$$

Важно различать: $p(\mathcal{D} \mid \theta)$ — это функция от θ при фиксированных данных $\mathcal{D}$ (то есть *правдоподобие*), а не вероятность θ.
Оценка максимального правдоподобия (MLE) выбирает такие параметры, при которых наблюдённые данные наиболее «вероятны»:

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} p(\mathcal{D} \mid \theta).$$

5.2 Следствия свойств i.i.d.

Часто используется предположение, что наблюдения являются i.i.d. (independent and identically distributed), то есть независимы и одинаково распределены:

$$z_1, z_2, \dots, z_n \ \text{ i.i.d.}$$

Тогда вероятность (плотность) всего датасета раскладывается в произведение по объектам:

$$p(\mathcal{D} \mid \theta) = p(z_1, \dots, z_n \mid \theta) = \prod_{i=1}^{n} p(z_i \mid \theta).$$

Далее часто переходят к логарифму правдоподобия (лог-правдоподобию), потому что с суммой работать проще, чем с произведением:

$$\log p(\mathcal{D} \mid \theta) = \log \prod_{i=1}^{n} p(z_i \mid \theta) = \sum_{i=1}^{n} \log p(z_i \mid \theta).$$

Логарифм — монотонно возрастающая функция, поэтому точки максимума сохраняются:

$$\arg\max_{\theta} p(\mathcal{D} \mid \theta) = \arg\max_{\theta} \log p(\mathcal{D} \mid \theta).$$

5.3 Формула отрицательного лог-правдоподобия

Так как в оптимизации часто минимизируют функцию, задачу MLE переписывают в виде минимизации **отрицательного лог-правдоподобия** (Negative Log-Likelihood, NLL):

$$\hat{\theta}_{\mathrm{MLE}} = \arg\min_{\theta} \big(-\log p(\mathcal{D} \mid \theta)\big).$$

С учётом i.i.d.-разложения получаем стандартную форму NLL как сумму по объектам:

$$\mathrm{NLL}(\theta) = -\log p(\mathcal{D} \mid \theta) = -\sum_{i=1}^{n} \log p(z_i \mid \theta) = -\sum_{i=1}^{n} \log p(x_i, y_i \mid \theta).$$

В задачах обучения с учителем NLL используется как функция потерь, по которой обучаются параметры модели.

6 Таксономия задач ML

6.1 Обучение с учителем и без учителя, примеры

Supervised learning: датасет $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, цель — построить $f(x;\theta)$ для предсказания y по x. Примеры: регрессия (цена, спрос), классификация (спам, дефект), ранжирование.
Unsupervised learning: датасет $\mathcal{D} = \{x_i\}_{i=1}^{n}$ *без* таргета. Цель — выявить структуру. Примеры: кластеризация, снижение размерности, поиск аномалий.

6.2 Назвать другие категории

Semi-supervised: часть размечена, часть нет. **Self-supervised:** псевдоразметка из данных. **Reinforcement learning:** агент максимизирует награду. **Generative:** моделирование $p(x)$ для генерации.

6.3 Практические примеры (минимум 3 для каждого типа)

Supervised: кредитный скоринг, прогноз спроса, детекция дефектов.
Unsupervised: сегментация клиентов, снижение размерности, поиск аномалий.
Semi-supervised: классификация изображений/текстов при дефиците разметки, детекция дефектов по сенсорам.
Self-supervised: предобучение языковых моделей (MLM), предобучение для изображений (masked patches, contrastive), предобучение на логах.
Reinforcement learning: управление ресурсами, робототехника, оптимизация стратегий.
Generative: генерация текста/изображений/табличных данных, восстановление/улучшение качества.

6.4 Постановка практической задачи, основные сущности

Датасет: $\mathcal{D} = \{(x_i, y_i)\}$ (supervised) или $\{x_i\}$ (unsupervised). **Признаки:** $X \in \mathbb{R}^{n \times d}$. **Модель:** $\hat{y} = f(X;\theta)$. **Loss:** оптимизируемая величина (MSE, log-loss). **Метрика:** оценка качества (может отличаться от loss). **Параметры θ vs гиперпараметры:** θ обучаются, гиперпараметры задаются. **Разбиение:** train/val/test. **Setup:** Problem + Dataset + Model + Metric/Loss.

7 Наивный байесовский классификатор

7.1 Формула Байеса

Байесовский подход рассматривает вероятностные распределения как центральный объект анализа. Базовым инструментом является **формула Байеса**, связывающая апостериорную вероятность класса с априорной вероятностью и правдоподобием наблюдений:

$$p(y \mid x) = \frac{p(x \mid y)\, p(y)}{p(x)}$$

В задаче классификации по признаковому вектору x требуется выбрать класс y с максимальной апостериорной вероятностью:

$$\hat{y} = \arg\max_{y \in \{1,\dots,K\}} p(y \mid x) = \arg\max_{y \in \{1,\dots,K\}} \frac{p(x \mid y)\, p(y)}{p(x)}$$

Поскольку знаменатель $p(x)$ не зависит от класса y, при поиске максимума его можно отбросить:

$$\hat{y} = \arg\max_{y \in \{1,\dots,K\}} p(x \mid y)\, p(y).$$

7.2 Основные предположения

Наивный байесовский классификатор (Naïve Bayes) вводит ключевое **наивное предположение** о том, что признаки $x_1, \dots, x_d$ считаются **условно независимыми** при фиксированном классе y.

$$x = (x_1, \dots, x_d), \qquad p(x \mid y) = \prod_{j=1}^{d} p(x_j \mid y),$$

Тогда правило классификации принимает вид:

$$\hat{y} = \arg\max_{y \in \{1,\dots,K\}} \left(p(y) \prod_{j=1}^{d} p(x_j \mid y) \right).$$

На практике для устойчивости вычислений часто переходят к логарифмам (лог-вероятностям), что заменяет произведение на сумму:

$$\hat{y} = \arg\max_{y \in \{1,\dots,K\}} \left(\log p(y) + \sum_{j=1}^{d} \log p(x_j \mid y) \right).$$

Оценки вероятностей $p(y)$ и условных распределений $p(x_j \mid y)$ получают по обучающей выборке (частотными оценками или параметрическими моделями), после чего применяют правило максимума апостериорной вероятности (MAP в смысле выбора класса).

7.3 Случай непрерывных признаков

Если признаки непрерывные, то вероятности вида $p(x_j \mid y)$ заменяются **плотностями** $p(x_j \mid y)$, и для каждого признака задаётся параметрическое семейство распределений, параметры которого оцениваются по обучающей выборке внутри каждого класса.
Типичные варианты параметризации (используются в стандартных реализациях Naïve Bayes):

- **Gaussian Naïve Bayes:** $x_j \mid (y = c) \sim \mathcal{N}(\mu_{jc}, \sigma_{jc}^2)$,

$$p(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2} \right).$$

- **Multinomial Naïve Bayes:** для счётчиков/частот (например, bag-of-words в текстах), где x_j – количество событий/слов.
- **Bernoulli Naïve Bayes:** для бинарных признаков $x_j \in \{0, 1\}$.

В общем случае при непрерывных признаках решающее правило остаётся тем же:

$$\hat{y} = \arg\max_{y \in \{1,\dots,K\}} \left(p(y) \prod_{j=1}^{d} p(x_j \mid y) \right),$$

но величины $p(x_j \mid y)$ интерпретируются как плотности и вычисляются через выбранную параметрическую модель (с параметрами, оценёнными по обучающим данным внутри класса).

8 Библиотека sklearn

8.1 Базовые классы, их функции и использование

scikit-learn (sklearn) — одна из самых крупных библиотек, объединяющих в одном месте множество методов машинного обучения и стандартных инструментов для подготовки данных, обучения, валидации и оценки моделей. В учебных и прототипных задачах sklearn удобен благодаря единообразному API, однако в индустриальном production-окружении часто выбирают специализированные решения (в т.ч. из-за требований к производительности, распределённости, мониторингу и т.п.).
Единый интерфейс API в sklearn построен вокруг нескольких базовых ролей (классов/абстракций). На практике важнее всего запомнить контракт методов: `fit`, `predict`, `transform` и их сочетания.

1. **Estimator (оценивать) / модель).**
Любой объект sklearn, который можно *обучить*, реализует метод:
`fit(X, y=None)`
где X — матрица признаков (обычно `shape = (n\_samples, n\_features)`), y — целевая переменная (опционально для unsupervised). Результат `fit` — обученный объект с сохранёнными параметрами (например, `coef_`, `classes_`, `feature_importances_` и др., в зависимости от модели).
2. **Predictor (предиктор): Classifier / Regressor.**
Модели для задач с учителем (классификация/регрессия) после `fit` используют:
`predict(X)`
Классификаторы часто дополнительно предоставляют:
`predict_proba(X)`, `decision_function(X)`
что позволяет работать с вероятностями, порогами и метриками (ROC-AUC, PR-AUC и др.).
3. **Transformer (преобразователь признаков).**
Преобразователь обучается на данных и затем применяет преобразование:
`fit(X, y=None)`, `transform(X)`
Часто используется объединённый метод:
`fit_transform(X, y=None)`
Примеры типовых трансформеров: масштабирование числовых признаков, кодирование категориальных признаков, снижение размерности и т.п.
4. **Model selection и evaluation (валидация и оценка).**
В sklearn есть стандартные инструменты для:
   - разбиения данных на train/val/test (например, `train_test_split`);
   - кросс-валидации (например, `cross_val_score`);
   - подбора гиперпараметров (например, `GridSearchCV`, `RandomizedSearchCV`);
   - вычисления метрик качества (accuracy, precision/recall/F-score, ROC-AUC и т.п.).
5. **Композиция моделей: Pipeline / ColumnTransformer.**
Практика показывает, что подготовку данных и модель нужно объединять в единый объект, чтобы корректно обучать и валидировать весь процесс целиком.

Минимальный шаблон использования sklearn (обучение и предсказание):

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
# (опционально) y_prob = model.predict_proba(X_test)[:, 1]
```

9 Регрессионная модель

9.1 Предположения

Регрессионная модель описывает наблюдаемую целевую величину как сумму *систематической (предсказуемой)* компоненты и *случайной* ошибки:

$$Y = f(X) + \varepsilon,$$

где $f(X)$ – детерминированная часть сигнала (то, что мы хотим предсказывать), а ε — шум (ошибка), отражающий вариативность, не объяснённую моделью.
Часто $f(X)$ интерпретируется как условное математическое ожидание:

$$f(X) = \mathbb{E}[Y \mid X], \qquad \varepsilon = Y - \mathbb{E}[Y \mid X],$$

то есть мы раскладываем наблюдение Y на:

- компоненту с **ожиданием**, которую можно предсказывать ($\mathbb{E}[Y \mid X]$);
- компоненту с **нулевым ожиданием** (шумы), которую предсказывать невозможно (ε).

Классические предположения регрессионной модели (underlying assumptions):

1. **Репрезентативности выборки.**
Наблюдения в датасете должны быть репрезентативны относительно генеральной совокупности, на которую планируется распространять выводы модели.
2. **Признаки измерены без ошибки.**
Независимые переменные (ковариаты) считаются измеренными без ошибок.
3. **Нулевое условное среднее ошибки (экзогенность).**
Отклонения от модели имеют нулевое ожидаемое значение при фиксированных ковариатах:

$$\mathbb{E}[\varepsilon \mid X_i] = 0.$$

4. **Гомоскедастичность.**
Дисперсия ошибок постоянна по наблюдениям:

$$\mathrm{Var}(\varepsilon_i \mid X_i) = \sigma^2 \quad \text{(не зависит от } i\text{)}.$$

5. **Отсутствие корреляции ошибок (нет автокорреляции).**
Остатки некоррелированы между собой, то есть ковариационная матрица ошибок диагональна:

$$\mathrm{Cov}(\varepsilon_i, \varepsilon_j) = 0 \ \text{ при } i \neq j.$$

9.2 * Почему это важно

Перечисленные предположения важны, потому что они обеспечивают корректные теоретические свойства оценивания и интерпретации регрессионной модели.

- **Корректность оценки зависимости $f(X)$.**
Условие $\mathbb{E}[\varepsilon_i \mid X_i] = 0$ является ключевым: оно означает, что в ошибке нет систематической составляющей, связанной с признаками. При нарушении (например, пропущенные важные факторы, эндогенность) оценки параметров становятся смещёнными, а предсказания — систематически неверными.
- **Гарантии оптимальности среди линейных несмещённых оценок (Gauss–Markov)**
При выполнении стандартных условий (в т.ч. гомоскедастичность и некоррелированность ошибок) оценка, получаемая минимизацией MSE в линейной регрессии, обладает свойством BLUE (Best Linear Unbiased Estimator): это несмещённая линейная оценка с минимальной дисперсией среди всех линейных несмещённых оценок.
- **Корректная оценка неопределённости и сравнение моделей.**
Гомоскедастичность и отсутствие корреляции ошибок лежат в основе корректных формул для дисперсий стандартных ошибок оценок параметров. Если, например, ошибки гетероскедастичны или коррелированы, то наивные оценки дисперсий будут неверны, что приводит к ошибочным выводам о значимости признаков и неверным доверительным интервалам.
- **Качество обобщения на целевую популяцию.**
Репрезентативность выборки критична для перехода результатов модели: если train-данные систематически отличаются от реальной популяции (смещение выборки), то качество на практике может резко падать даже при отличной метрике на обучении/валидации.
- **Практические последствия нарушений.**
Ошибка измерения признаков приводит к искажению оценок (вплоть до систематического занижения эффектов), коррелированные ошибки делают модель пересуженной, а неоднородная дисперсия ошибок снижает эффективность оценивания и ухудшает интерпретацию.

10 Линейная регрессия

10.1 Мотивация функции потерь MSE

Линейная модель: $\hat{y}_i = x_i^\top w$. Постановка: $y_i = x_i^\top w + \varepsilon_i$.
MSE: $\mathrm{MSE}(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - x_i^\top w)^2 = \frac{1}{n}\|y - Xw\|_2^2$.

10.1.1 Доказательство для гауссового случая

Если $\varepsilon_i \overset{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2)$, то $y_i \mid x_i \sim \mathcal{N}(x_i^\top w, \sigma^2)$.
Лог-правдоподобие: $\log p(y \mid X, w, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum(y_i - x_i^\top w)^2$.
$\arg\max_w \log p = \arg\min_w \mathrm{MSE}(w) \to$ MLE приводит к MSE.

10.2 Аналитическое решение для весов (матричная форма)

$J(w) = \|y - Xw\|_2^2$. Градиент: $\nabla_w J = -2X^\top y + 2X^\top Xw$. Нормальные уравнения: $X^\top Xw = X^\top y$.
Решение (при $X^\top X$ обратимы): $\hat{w} = (X^\top X)^{-1} X^\top y$
Если вырождена: $\hat{w} = X^+ y$ (псевдообратная).

10.2.1 Доказательство

$J(w) = y^\top y - 2w^\top X^\top y + w^\top X^\top Xw$. Градиент: $\nabla_w J = -2X^\top y + 2X^\top Xw$. Приравнив к нулю: $X^\top Xw = X^\top y \Rightarrow \hat{w} = (X^\top X)^{-1} X^\top y$. Так как $J(w)$ является выпуклой квадратичной функцией, найденная стационарная точка является глобальным минимумом.

10.3 Теорема Гаусса–Маркова, BLUE

Рассмотрим классическую линейную модель:

$$y = X\beta + \varepsilon,$$

где $\beta \in \mathbb{R}^d$ — истинный вектор параметров, а ошибки удовлетворяют условиям:

$$\mathbb{E}[\varepsilon \mid X] = 0, \qquad \mathrm{Var}(\varepsilon \mid X) = \sigma^2 I_n.$$

Предположим также, что X имеет полный столбцовый ранг, так что $(X^\top X)^{-1}$ существует.
Оценка МНК (OLS):

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

Теорема Гаусса–Маркова: при указанных предположениях оценка $\hat{\beta}$ является **BLUE** (Best Linear Unbiased Estimator), то есть:

- **линейная:** $\hat{\beta} = Ay$ для некоторой матрицы A;
- **несмещённая:** $\mathbb{E}[\hat{\beta} \mid X] = \beta$;
- **наилучшая по дисперсии:** среди всех линейных несмещённых оценок имеет минимальную ковариационную матрицу (в смысле неотрицательной определённости разности).

10.3.1 * Доказательство

Шаг 1: линейность и несмещённость OLS. OLS имеет вид $\hat{\beta} = Ay$, где

$$A = (X^\top X)^{-1} X^\top.$$

Тогда

$$\mathbb{E}[\hat{\beta} \mid X] = \mathbb{E}[Ay \mid X] = A\mathbb{E}[X\beta + \varepsilon \mid X] = AX\beta + A\mathbb{E}[\varepsilon \mid X]) = AX\beta = \beta,$$

поскольку

$$AX = (X^\top X)^{-1} X^\top X = I_d.$$

Значит, OLS несмещённа.
Шаг 2: ковариация OLS.

$$\mathrm{Var}(\hat{\beta} \mid X) = \mathrm{Var}(Ay \mid X) = A\,\mathrm{Var}(y \mid X)\,A^\top = A\,\mathrm{Var}(\varepsilon \mid X)\,A^\top = \sigma^2 AA^\top = \sigma^2 (X^\top X)^{-1}.$$

Шаг 3: сравнение с произвольной линейной несмещённой оценкой. Пусть $\tilde\beta = By$ — любая линейная несмещённая оценка. Несмещённость означает:

$$\mathbb{E}[\tilde\beta \mid X] = \beta \iff BX = I_d.$$

Рассмотрим разность $B - A$ и обозначим $C := B - A$. Тогда

$$CX = (B - A)X = BX - AX = I_d - I_d = 0.$$

Ковариация $\tilde\beta$:

$$\mathrm{Var}(\tilde\beta \mid X) = \sigma^2 BB^\top = \sigma^2(A + C)(A + C)^\top = \sigma^2(AA^\top + AC^\top + CA^\top + CC^\top).$$

Покажем, что смешанные члены равны нулю. Заметим:

$$AX = (X^\top X)^{-1}X^\top X = I_d, \qquad CX = 0.$$

Отсюда следует

$$AC^\top = (CA^\top)^\top.$$

При стандартном выводе теоремы Гаусса–Маркова используется факт ортогональности остатка к пространству столбцов X в МНК-проекции, что эквивалентно занулению смешанных членов, и в итоге получается представление:

$$\mathrm{Var}(\tilde\beta \mid X) = \sigma^2(AA^\top + CC^\top) = \mathrm{Var}(\hat\beta \mid X) + \sigma^2 CC^\top.$$

Так как матрица $CC^\top$ неотрицательно определена, то

$$\mathrm{Var}(\tilde\beta \mid X) - \mathrm{Var}(\hat\beta \mid X) = \sigma^2 CC^\top \succeq 0.$$

Следовательно, никакая другая линейная несмещённая оценка не может иметь меньшую ковариационную матрицу, чем OLS. Значит, $\hat\beta$ является BLUE.

- **порождает разреженность**: многие коэффициенты становятся нулевыми;
- выполняет **неявный отбор признаков** (feature selection);
- **недифференцируема** в нуле, но остаётся выпуклой; оптимизация требует специметодов;
- решение также **смещённое** (biased), как и при любой регуляризации.

Elastic Net:

- сочетает **устойчивость Ridge** и **разреженность Lasso**;
- часто предпочтителен при наличии **групп коррелированных признаков**.

В целом регуляризация управляет компромиссом **bias–variance**: обычно увеличивает смещение, но снижает дисперсию оценки и чувствительность к шуму/выбросам, что улучшает обобщающую способность модели.

- **Линейная регрессия:** тип регуляризации и коэффициент регуляризации (например, L_1, L_2, значение λ). :contentReference[oaicite:8]|index=8
- **Деревья решений:** максимальная глубина дерева (`max_depth`) и другие ограничения роста дерева.
- **Нейросети:** число слоёв (архитектурная настройка). :contentReference[oaicite:10]|index=10
- **RNN (пример из PyTorch-постановки):** `input_size`, `hidden_size`, `num_layers`, `dropout`, `bidirectional`. :contentReference[oaicite:11]|index=11

Примеры параметров (что именно обучается):

- **Линейная регрессия / линейные модели:** веса w (и, при наличии, свободный член w_0).
- **Деревья решений:** фактически выбранные разбиения (признаки и пороги) и значения в листьях являются «внутренними» настроенными элементами модели при фиксированном наборе гиперпараметров. :contentReference[oaicite:12]|index=12
- **Нейросети:** матрицы весов и векторы смещений слоёв, которые оптимизируются градиентными методами.

- **Разные вероятностные модели и интерпретации.** Например, логистическая функция потерь выводится из MLE при бернуллиевской модели вероятности класса; а то время как hinge loss соответствует максимизации зазора (margin) в SVM-подходе.

Типовые примеры surrogate losses (как функций от маржи $m = y\,s(x)$):

- **Square loss (почти не используют для классификации):**
$$\ell(m) = (1 - m)^2.$$
- **Logistic loss (логистическая регрессия):**
$$\ell(m) = \log\bigl(1 + \exp(-m)\bigr).$$
- **Hinge loss (SVM):**
$$\ell(m) = \max(0, 1 - m).$$
- **Exponential loss (исторически важна для бустинга):**
$$\ell(m) = \exp(-m).$$

Выбор конкретной функции потерь — это выбор компромисса между оптимизационными свойствами, геометрическими (маржинальными) свойствами и вероятностной интерпретацией модели.

11 Градиентный спуск для линейной регрессии

11.1 Почему не используют матричное решение

Для линейной регрессии с MSE существует аналитическое решение (через нормальные уравнения) однако на практике в большинстве реальных задач предпочитают итеративные методы оптимизации (градиентный спуск и его варианты). Основные причины:

- **Вычислительная сложность.** Аналитическое решение требует вычисления и (явно или неявно) обращения матрицы $X^\top X$:
$$\hat w = (X^\top X)^{-1}X^\top y.$$
При d признаках обращение матрицы размера $d \times d$ имеет кубическую сложность по размеру (порядка $O(d^3)$), а также требуется вычислить $X^\top X$ (порядка $O(nd^2)$) и хранить промежуточные матрицы. Для больших d или при дорогих операциях ввода/вывода это становится узким местом.
- **Численная неустойчивость (плохая обусловленность) и мультиколлинеарность.** Если признаки сильно коррелированы, то матрица $X^\top X$ становится близкой к вырожденной (почти сингулярной). Тогда обращение (или решение нормальных уравнений) становится численно нестабильным: небольшие погрешности вычислений (в том числе из-за конечной точности арифметики) приводят к большим ошибкам в оценке весов, возможны «огромные» коэффициенты, которые компенсируют друг друга.
- **Память и инженерные ограничения.** Для больших датасетов X может не помещаться в память; аналитическое решение предполагает глобальную обработку данных. Итеративные методы допускают обучение по батчам/мини-батчам и потоковую обработку данных, что проще реализовать в распределённых и производственных системах.
- **Единый подход для разных моделей.** Даже если для линейной регрессии есть closed-form, для большинства моделей (логистическая регрессия, нейросети, многие регуляризованные постановки и т.п.) аналитического решения нет, и всё равно используются градиентные методы. Поэтому на практике часто сразу применяют унифицированный оптимизационный пайплайн.

Напомним формулировку задачи (MSE):

$$\min_{w\in\mathbb{R}^d} J(w), \qquad J(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - x_i^\top w)^2 = \frac{1}{n}\|y - Xw\|_2^2.$$

Полный (batch) градиент MSE в матричной форме:

$$\nabla_w J(w) = \frac{2}{n}X^\top(Xw - y).$$

Шаг градиентного спуска:

$$w^{(t+1)} := w^{(t)} - \eta\,\nabla_w J\bigl(w^{(t)}\bigr),$$

где $\eta > 0$ — шаг обучения (learning rate).

11.2 Стохастический градиентный спуск, преимущества

Стохастический градиентный спуск (SGD) — вариант градиентного спуска, в котором градиент оценивается не по всему датасету, а по случайному подмножеству объектов.
SGD по одному объекту. Для одного наблюдения (x_i, y_i) определим стохастическую оценку градиента:

$$g_i(w) = \nabla_w\Bigl((y_i - x_i^\top w)^2\Bigr) = 2(x_i^\top w - y_i)x_i.$$

Тогда шаг SGD:

$$w^{(t+1)} = w^{(t)} - \eta_t\, g_{i_t}\bigl(w^{(t)}\bigr),$$

где i_t — случайно выбранный индекс объекта на шаге t.
Mini-batch SGD. Чаще используют мини-батчи $B_t \subset \{1, \dots, n\}$ размера $|B_t| = b$:

$$g_{B_t}(w) = \frac{2}{b}X_{B_t}^\top(X_{B_t}w - y_{B_t}), \qquad w^{(t+1)} = w^{(t)} - \eta_t\, g_{B_t}\bigl(w^{(t)}\bigr).$$

Преимущества SGD (и mini-batch SGD):

- **Масштабируемость по числу объектов** n. Один шаг SGD/mini-batch SGD стоит порядка $O(bd)$ (или $O(d)$ при $b = 1$), тогда как полный градиент требует прохода по всем n объектам. Это особенно важно при очень больших датасетах.
- **Меньшие требования к памяти и удобство потокового обучения.** Для обновления весов достаточно держать в памяти один объект или мини-батч, что позволяет обучаться на данных, которые не помещаются в RAM, и обновлять модель по мере поступления данных.
- **Быстрое получение «достаточно хорошего» решения.** На первых итерациях SGD часто даёт заметный прирост качества быстрее, чем методы, требующие полной обработки датасета на каждом шаге. Это полезно для быстрых baseline и прототипирования.
- **Естественная совместимость с батчевой обработкой и параллелизмом.** Mini-batch SGD хорошо ложится на векторизацию и ускорение на GPU/TPU, а также на распределённые схемы (обработка батчей на нескольких устройствах).
- **Сходимость в выпуклом случае.** Для линейной регрессии (выпуклая квадратичная задача) SGD при корректном выборе расписания шагов η_t сходится к оптимальному решению (в смысле ожидания/по вероятности).

12 Регуляризация в линейных моделях

12.1 Мотивация регуляризации

Проблема: при мультиколлинеарности $X^\top X$ плохо обусловлена → неустойчивое решение (большие веса). Регуляризация: $J_{\text{reg}}(w) = J(w) + \lambda\Omega(w)$, где $\Omega(w)$ — штраф (норма весов), $\lambda \geq 0$ — гиперпараметр. Решение становится более устойчивым, но смещённым.

12.2 Гребневая регрессия Ridge (L2)

12.2.1 Формула решения

Ridge: $J_{\text{Ridge}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda\|w\|_2^2$.
Градиент: $\nabla_w J = \frac{2}{n}X^\top(Xw - y) + 2\lambda w = 0 \Rightarrow (X^\top X + n\lambda I)w = X^\top y$.
Решение: $\boxed{w_{\text{Ridge}} = (X^\top X + \lambda I)^{-1}X^\top y}$, где $\tilde\lambda = n\lambda$.
Диагональная добавка делает систему невырожденной. Свободный член обычно не регуляризуют: $\Omega(w) = \|w_{1:d}\|$.

12.3 Lasso-регуляризация (L1)

Lasso: $J_{\text{Lasso}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda\|w\|_1$, $\|w\|_1 = \sum|w_j|$.
Особенности: разреженное решение (многие $w_j = 0$), незвный отбор признаков, недифференцируем в нуле → оптимизация: субградиентный спуск, субградиенты/проксимальные методы. Нет простой замкнутой формулы (кроме частных случаев: soft-thresholding для ортонормированного дизайна).

12.4 Вероятностная интерпретация

Регуляризация как MAP: $\hat w_{\text{MAP}} = \arg\max_w p(w \mid X, y) = \arg\min_w(-\log p(y \mid X, w) - \log p(w))$.
Ridge: $w \sim \mathcal{N}(0, \tau^2 I) \Rightarrow -\log p(w) \propto \|w\|_2^2 \to L_2$-регуляризация.
Lassо: $p(w) \propto \exp(-a\|w\|_1) \Rightarrow -\log p(w) \propto \|w\|_1 \to L_1$-регуляризация.

12.5 Elastic Net

Elastic Net сочетает L_1 и L_2 штрафы, объединяя свойства Ridge и Lasso:

$$J_{\text{EN}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda_1\|w\|_1 + \lambda_2\|w\|_2^2.$$

Эквивалентная параметризация часто задаётся через общий коэффициент λ и долю L_1:

$$J_{\text{EN}}(w) = \frac{1}{n}\|y - Xw\|_2^2 + \lambda\left(\alpha\|w\|_1 + (1-\alpha)\|w\|_2^2\right), \qquad \alpha \in [0, 1].$$

Elastic Net полезен, когда:

- хочется **разреженности** (как у Lasso), но при этом
- есть **сильная корреляция признаков** (где чистый Lasso может быть нестабилен), а L_2-компонента стабилизирует решение.

12.6 Свойства регуляризаций

L_2 **(Ridge):**

- **сжимает веса** (shrinkage), особенно большие по модулю;
- **стабилизирует** решение и улучшает обусловленность (важно при мультиколлинеарности);
- **дифференцируема** и удобна для градиентных методов;
- обычно **не даёт строго нулевых** весов (признаки сохраняются, но их вклад уменьшается);
- решение **смещённое** (biased), то есть не обязано сходиться к w^* при фиксированном $\lambda > 0$.

L_1 **(Lasso):**

13 Метрики регрессии MSE, MAE, RMSE, MAPE, SMAPE, R2

13.1 Когда их используют

Пусть даны истинные значения $y_1, \dots, y_n$ и предсказания модели $\hat y_1, \dots, \hat y_n$. Обозначим ошибки $e_i = y_i - \hat y_i$ и среднее $\bar y = \frac{1}{n}\sum_{i=1}^n y_i$. В лекции выделяются базовые метрики регрессии: MSE, MAE, RMSE, MAPE, SMAPE, R^2. :contentReference[oaicite:0]|index=0

- **MSE (Mean Squared Error).**
$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^n (y_i - \hat y_i)^2 = \frac{1}{n}\sum_{i=1}^n e_i^2.$$
Когда использовать:
  - когда важно **сильно штрафовать большие ошибки** (квадрат усиливает вклад крупных промахов);
  - как **оптимизируемую loss-функцию** (гладкая, дифференцируемая), в т.ч. для градиентных методов;
  - при вероятностной мотивации через **гауссовский шум** (MLE приводит к MSE) и в контексте классической линейной регрессии.

  Ограничение: чувствительна к выбросам/тяжёлым хвостам (outliers). :contentReference[oaici
- **RMSE (Root Mean Squared Error).**
$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^n (y_i - \hat y_i)^2}.$$
Когда использовать:
  - когда нужна метрика **в тех же единицах измерения**, что и таргет (в отличие от MSE, которая в квадрате единиц);
  - когда важно сохранить свойства MSE (сильный штраф больших ошибок), но сделать интерпретацию более «прикладной».
- **MAE (Mean Absolute Error).**
$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^n |y_i - \hat y_i| = \frac{1}{n}\sum_{i=1}^n |e_i|.$$
Когда использовать:
  - когда нужна **робастность** к выбросам (линейный штраф, в отличие от квадратичного);
  - когда важнее «средняя абсолютная ошибка», а не редкие крупные промахи;
  - как более устойчивый критерий при негаусссовском шуме или тяжёлых хвостах. :contentReference[oaicite:2]|index=2

  Замечание: MAE недифференцируема в нуле, но на практике это решаемо (субградиенты, проксимальные методы и т.п.). :contentReference[oaicite:3]|index=3
- **MAPE (Mean Absolute Percentage Error).**
$$\mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^n \left|\frac{y_i - \hat y_i}{y_i}\right| = \frac{100\%}{n}\sum_{i=1}^n \left|\frac{e_i}{y_i}\right|.$$
Когда использовать:
  - когда важно измерить ошибку **в относительных величинах** (проценты), чтобы сравнивать качество на разных масштабах таргета;
  - в бизнес-постановках, где «ошибка в процентах» напрямую интерпретируема (например прогноз спроса/выручки).

Ограничения:

  - не определена при $y_i = 0$ и становится крайне нестабильной при $|y_i| \approx 0$;
  - асимметрична по отношению к завышению/занижению через деление именно на y_i.
- **SMAPE (Symmetric Mean Absolute Percentage Error).** Один из распространённых вариантов:
$$\mathrm{SMAPE} = \frac{100\%}{n}\sum_{i=1}^n \frac{2|y_i - \hat y_i|}{|y_i| + |\hat y_i|}$$
Когда использовать:
  - когда нужна **относительная метрика**, но хочется уменьшить асимметрию MAPE;
  - когда в данных встречаются малые значения таргета, и требуется более «мягкая» нормировка (через сумму модулей).

Ограничения:

  - при $|y_i| + |\hat y_i| = 0$ выражение требует аккуратного определения (обычно задают 0 по соглашению);
  - интерпретация «в процентах» зависит от выбранной формулы (в литературе встречаются близкие варианты).
- R^2 **(коэффициент детерминации).**
$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat y_i)^2}{\sum_{i=1}^n (y_i - \bar y)^2} = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}}.$$
Когда использовать:
  - когда нужно понять, какую долю вариации таргета объясняет модель **по сравнению с базовой константной моделью** $\hat y_i = \bar y$;
  - для отчётности и сопоставления моделей на **одном и том же датасете** (как безразмерная метрика).

Особенности:

  - R^2 может быть **отрицательным**, если модель хуже предсказания средним;
  - при отсутствии свободного члена и/или при специфических протоколах оценки возможны инсайты интерпретации.

Практическое правило выбора.

- Если критичны большие ошибки и нужна удобная оптимизация: чаще берут **MSE**/**RMSE**.
- Если важна устойчивость к выбросам и «типичная» ошибка: берут **MAE**.
- Если качество должно быть сопоставимо в процентах (и y не близок к нулю): берут **MAPE**/**SMAPE**.
- Если нужна интерпретация «насколько лучше среднего»: используют R^2 (обычно вместе с одной из etor-метрик).

15 Валидация и оценка модели

15.1 Недообучение и переобучение, определения и способы обнаружения

Недообучение (underfitting) — модель слишком простая (высокий bias), не аппроксимирует зависимость даже на train. Картина: ошибка на train высокая и ошибка на val/test высокая.
Переобучение (overfitting) — модель подстраивается под шум train (высокий variance). Картина: ошибка на train низкая, на val/test существенно выше (большой generalization gap).
Способы обнаружения:

- Сравнение метрик train vs val/test.
- Кривые обучения: зависимость метрики от итераций/размера выборки.
- Валидационные кривые по гиперпараметрам (глубина, λ, итерации).

Борьба:

- underfitting: усложнить модель, улучшить признаки, ослабить регуляризацию, обучать дольше.
- overfitting: усилить регуляризацию, early stopping, увеличить данные, использовать ансамбли.

15.2 Способы разбиения выборки

15.2.1 Что такое хорошее разбиение, идеальный и практический варианты

Хорошее разбиение: совпадение распределений (на практике — ключевых статистик) между train/val/test, репрезентативность относительно будущих данных, отсутствие утечек, учёт временной/групповой структуры.

15.2.2 4 основных типа разбиений

1. **Равномерное случайное (uniform random):** объекты случайно распределяются. Для i.i.d. данных без зависимостей.
2. **Стратифицированное (stratified):** сохраняет пропорции классов. При дисбалансе классов
3. **Групповое (grouped):** все объекты группы целиком в train или val/test. Для коррелирова объектов (пациент, пользователь).
4. **Временное (timeseries):** train из прошлого, val/test из будущего. Для временных рядов (нельзя перемешивать).

15.3 Этапы построения модели

15.3.1 Train, validation, test, мотивация каждого

- **Train:** обучение параметров модели (веса w).
- **Validation:** выбор гиперпараметров, контроль переобучения, сравнение вариантов пайплай
- **Test:** финальная честная оценка после выбора всех решений. Нельзя использовать в цикле подбора.

Типовая ошибка: подбирать гиперпараметры по test — оценка становится оптимистически смещённой.

15.3.2 Кросс-валидация

k-fold CV: датасет делят на k фолдов, модель обучают k раз (каждый раз на $k-1$ фолдах, валидируя на оставшемся), агрегируют метрики. Плюс: устойчивые оценки на малых данных. Минус: высокая вычислительная стоимость.
Leave-One-Out (LOO): $k = n$ (валидация по одному объекту). Используется редко.

17 Логистическая регрессия

17.1 Базовое предположение и геометрическая интерпретация

Базовое предположение логистической регрессии: каждой точке пространства признаков $x \in \mathbb{R}^d$ соответствует **вероятность** принадлежности к положительному классу:

При этом логистическая регрессия задаёт параметрическое семейство вероятностей, связывая их с линейным скором (линейных предикторов):

$$z(x) = w^\top x + b, \qquad p(y = 1 \mid x) = \sigma(z(x)).$$

Эквивалентная формулировка через **логиты / log-odds**:

$$\log\frac{p(y = 1 \mid x)}{1 - p(y = 1 \mid x)} = w^\top x + b.$$

То есть **логарифм отношения шансов** (odds) является линейной функцией признаков.
Геометрическая интерпретация. Граница решений при пороге 0.5 соответствует условию

$$p(y = 1 \mid x) = 0.5 \iff z(x) = 0 \iff w^\top x + b = 0,$$

то есть это **гиперплоскость** в $\mathbb{R}^d$, как и у линейного классификатора. Вектор w является нормально к гиперплоскости, а b задаёт её сдвиг.
Классификационное правило по усл:

$$\hat y = \mathbb{I}\bigl[p(y = 1 \mid x) \geq 0.5\bigr] = \mathbb{I}\bigl[w^\top x + b \geq 0\bigr].$$

17.2 Сигмоида как предсказание вероятности, свойства

Сигмоида (логистическая функция) определяется как

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

В логистической регрессии предсказание вероятности положительного класса:

$$\hat p(x) = p(y = 1 \mid x) = \sigma(w^\top x + b).$$

Ключевые свойства сигмоиды:

- **Область значений:** $\sigma(z) \in (0, 1)$ для любого $z \in \mathbb{R}$, что делает её естественным преобразов "скор → вероятность".
- **Монотонность:** $\sigma(z)$ строго возрастает по z.
- **Симметрия:**
$$\sigma(-z) = 1 - \sigma(z),$$
то есть функция "симметрична" относительно точки $(0, 0.5)$.
- **Производная:**
$$\sigma'(z) = \sigma(z)(1 - \sigma(z)),$$
что важно для градиентных методов оптимизации.
- **Связь с logit:** обратное преобразование к σ есть логит
$$\mathrm{logit}(p) = \log\frac{p}{1 - p}, \qquad \mathrm{logit}\bigl(\sigma(z)\bigr) = z.$$

17.3 Формула логистической функции потерь

Рассмотрим бинарную классификацию с метками $y_i \in \{0, 1\}$ и предсказаниями вероятностей $\hat p_i = \sigma(w^\top x_i + b)$. Тогда стандартная **логистическая функция потерь**, (она же **log-loss, binary cross-entropy**) на датасете:

$$\mathcal{L}(w, b) = -\sum_{i=1}^n \Bigl[y_i \log\hat p_i + (1 - y_i)\log(1 - \hat p_i)\Bigr].$$

Часто используют усреднённый вариант $\frac{1}{n}\mathcal{L}$.

Эквивалентная форма через маржу (удобна при метках $y_i \in \{-1, +1\}$). Пусть $y_i \in \{-1, +1\}$ и $z_i = w^\top x_i + b$. Тогда

$$\mathcal{L}(w, b) = \sum_{i=1}^n \log\bigl(1 + \exp(-y_i z_i)\bigr).$$

17.3.1 вывод через MLE

Пусть задана вероятностная модель: для каждого объекта (x_i, y_i)

$$y_i \mid x_i \sim \mathrm{Bernoulli}(p_i), \qquad p_i = \sigma(w^\top x_i + b).$$

Тогда правдоподобие выборки (при независимости объектов) равно

$$p(y \mid X, w, b) = \prod_{i=1}^n p_i^{y_i}(1 - p_i)^{1 - y_i}.$$

Лог-правдоподобие:

$$\log p(y \mid X, w, b) = \sum_{i=1}^n \Bigl[y_i \log p_i + (1 - y_i)\log(1 - p_i)\Bigr].$$

MLE-оценка максимизирует лог-правдоподобие, то есть эквивалентно минимизирует отрицатель лог-правдоподобие:

$$(\hat w, \hat b) = \arg\max_{w,b} \log p(y \mid X, w, b) = \arg\min_{w,b}\left(-\sum_{i=1}^n \Bigl[y_i \log p_i + (1 - y_i)\log(1 - p_i)\Bigr]\right).$$

Получаем, что минимизация логистической потери **совпадает с MLE** для Бернуллиевской модели, где параметр вероятности задаётся через сигмоиду от линейного скора.

17.4 Свойство калибровки вероятностей

Калибровочность вероятностей (calibration) в бинарном случае означает, что предсказанное значение $\hat p$ соответствует истинной доле положительного класса среди объектов с такими предска:

$$\mathbb{P}(Y = 1 \mid \hat p = p) = p.$$

Интерпретация: если модель многократно выдаёт вероятность 0.8 на группе объектов, то примерно в 80% случаев класс действительно будет положительным.
Логистическая регрессия является **вероятностной моделью**: она непосредственно моделирует Бернуллиевское распределение и каждой точке пространства признаков, то есть выдаёт именно $\hat p(x) = p(y = 1 \mid x)$. Поэтому в типовом изложении логистическая регрессия рассматривается как **калиброванный классификатор**. На практике калибровка может ухудшаться при сильной misспецификации модели, сдвиге распределений или агрессивной регуляризации; тогда применяют отдельные процедуры калибровки, но базовая мотивация логистической регрессии — именно получение вероятностей, а не только меток.

17.5 Обобщённая линейная модель

Обобщённая линейная модель (GLM) задаётся тремя компонентами:

- распределение отклика Y из **экспоненциального семейства**;
- линейный предиктор $\eta(x) = w^\top x + b$;
- **link-функция** $g(\cdot)$, связывающая математическое ожидание отклика с линейным предикто
$$g(\mu(x)) = \eta(x), \qquad \mu(x) = \mathbb{E}[Y \mid X = x].$$

Логистическая регрессии — частный случай GLM для
$Y \mid X = x \sim \mathrm{Bernoulli}(p(x)), \qquad \mu(x) = \mathbb{E}[Y \mid X = x] = p(x),$
с **логит-ссылкой** (logit link):

$$g(p) = \log\frac{p}{1 - p}, \qquad g(p(x)) = w^\top x + b.$$

Эквивалентно:

$$p(x) = \sigma(w^\top x + b).$$

Таким образом, логистическая регрессия — это GLM, где распределение отклика Бернулли, а link-функция логит. что и приводит к сигмоиде в явной форме вероятности.

18 Стратегии многоклассовой классификации

18.1 One-vs-one, one-vs-all и их свойства

Пусть решается задача многоклассовой классификации:

$$y \in \{1, 2, \dots, K\}, \qquad \mathcal{D} = \{(x_i, y_i)\}_{i=1}^n.$$

Многие базовые классификаторы естественно устроены как бинарные, поэтому многоклассовую задачу часто сводят к набору бинарных задач и затем агрегируют решения.
One-vs-All / One-vs-Rest (OvA/OvR).
Идея: обучить K бинарных классификаторов $\{f_k\}_{k=1}^K$, где k-й классификатор отделяет класс k от "всех остальных":

$$\text{для } f_k: \quad \tilde y_i^{(k)} = \begin{cases} +1, & y_i = k, \\ -1, & y_i \neq k. \end{cases}$$

Если базовый классификатор линейный, то

$$f_k(x) = w_k^\top x + b_k,$$

а **маржа** равна $m_k(x) = y_k(x)$ (или $m_k(x) = y\,f_k(x)$ если записи с $\{-1, +1\}$).
Инференс (предсказание класса). Стандартное правило агрегации: взять класс с максимала "скорингом" (максимальной маржой):

$$\hat y(x) = \arg\max_{k \in \{1,\dots,K\}} f_k(x).$$

Если базовый метод выдаёт вероятности $\hat p_k(x) = \mathbb{P}(y = k \mid x)$ (или их суррогаты), то часто используют

$$\hat y(x) = \arg\max_k \hat p_k(x).$$

Свойства OvR:

- **Число обучаемых моделей:** K бинарных классификаторов.
- **Данные для каждого классификатора:** используется **весь датасет** (все n объектов), но метки переходят в "класс k против остальных".
- **Класс-имбаланс в подзадачах:** для каждого f_k положительный класс один, отрицатель — "все остальные", поэтому часто возникает сильный дисбаланс, особенно при больших K или редких классах.
- **Проблема сопоставимости скорингов:** значения $f_k(x)$ (маржи) от разных бинарных классификаторов могут быть **на разных шкалах**, поэтому простое arg max $f_k(x)$ может быть чувствительно к калибровке/масштабам.
- **Неоднозначность по порогам:** если вместо arg max пытаться решать через пороги (например, "берём все классы, где $f_k(x) \geq 0$"), то возможны случаи:
  - **несколько** классов одновременно "положительные";
  - **ни один** класс не "положительный".

  Поэтому на практике обычно выбирают именно максимум по скору/вероятности.

One-vs-One (OvO).
Идея: обучить бинарный классификатор для **каждой пары классов** (k, ℓ), $1 \leq k < \ell \leq K$:

$$f_{k\ell}: \text{ предиктит } k \text{ от } \ell \text{ и обучается только на объектах этих двух классов.}$$

При линейной модели:

$$f_{k\ell}(x) = w_{k\ell}^\top x + b_{k\ell}.$$

Инференс (голосование). Каждый парный классификатор "голосует" за один из двух классов, после чего суммируются голоса:

$$v_k(x) = \sum_{\ell \neq k} \mathbb{I}\bigl[f_{k\ell}(x) > 0\bigr] \quad (\text{с корректной интерпретацией знака для пары } (k, \ell)),$$

и итоговый класс выбирается по большинству голосов:

$$\hat y(x) = \arg\max_k v_k(x).$$

(Возможны ничьи; тогда применяют правила разрушения, например по сумме марж, по вероятностям или по приоритету классов.)
Свойства OvO:

- **Число обучаемых моделей:** $\frac{K(K-1)}{2}$ бинарных классификаторов.
- **Данные для каждого классификатора: подвыборка** датасета (только два класса), поэтому каждая подзадача обычно лучше сбалансирована, чем в OvR, и может быть проще для обучения.
- **Вычислительная цена:**
  - обучение требует много моделей (быстро растёт по K);
  - инференс также требует прогнать много моделей (порядка $O(K^2)$ бинарных предиктов)
- **Практический компромисс:** OvO часто выигрывает, когда K не очень велико, а бинарный классификатор чувствителен к дисбалансу или сильно зависит от "лишающих" классов; OvR часто выигрывает по простоте и скорости при большом K.

Короткое сравнение OvR и OvO:

$$\#\text{классификаторов: OvR} = K, \qquad \text{OvO} = \frac{K(K-1)}{2};$$

данные на классификатор: OvR = полный датасет, OvO = подвыборка по двум классам.

- **Связь с регуляризацией:** L_2-регуляризация штрафует $\|w\|_2^2$, а $L_1 - \|w\|_1$, что приводит к разным свойствам решений (сжатие vs разреженность).

3) Пропуски и выбросы: требуется импутация; при выбросах используют робастные методы.
4) Категориальные признаки: переводят в one-hot перед PCA.
Важно: параметры предобработки вычисляются только на train и применяются к val/test.

3. **Спуск по уровням.** Повторяют жадный поиск на каждом уровне сверху вниз, получая "приближение" к области x.
4. **Формирование рёбер на уровнях $\leq L(x)$.** На каждом уровне находят кандидатов (через локальный поиск с параметром efConstruction), выбирают до M соседей и добавляют рёбра между x и выбранными вершинами. *Часто применяют* **эвристику разнообразия** (diversification), чтобы не соединять только очень близкие (почти коллинеарные) точки, улучшая навигацию графа.

Ключевые гиперпараметры HNSW:

- M — максимальное число рёбер (соседей) на уровне для вершины;
- efConstruction — "ширина" поиска кандидатов при построении (влияет на качество графа и время построения);
- efSearch — "ширина" поиска при запросе (влияет на recall и latency).

24.5.2 Использование графа

Поиск k ближайших соседей для запроса q:

1. **Верхние уровни (быстрый навигатор).** Начинают с точки входа на самом верхнем уровне и выполняют **жадный спуск**: переходят к соседу, уменьшающему расстояние до q, пока улучшений нет; затем переходят на уровень ниже.
2. **Нижний уровень (точная локализация).** На уровне 0 выполняют **best-first** поиск с ограничением размера кандидатов efSearch: поддерживают множество кандидатов и множество лучших найденных, расширяют ближайшие кандидаты, пока критерий остановки не выполнен.
3. **Ответ.** Возвращают k ближайших среди найденных (по точной метрике).

Свойства HNSW (практические):

- очень хорошее отношение **качество (recall) / скорость** для высокоразмерных эмбеддингов
- параметр efSearch позволяет плавно регулировать trade-off: больше efSearch ⇒ выше recall, но выше latency;
- хорошо поддерживает **инкрементальные вставки** (в отличие от некоторых методов, ориентированных на офлайн построение).

24.6 Примеры векторных баз данных

Векторная БД (vector database) — система хранения эмбеддингов (векторов) и метаданных, которая предоставляет:

- поиск ближайших соседей (обычно ANN) по метрике (cosine / L2 / inner product);
- фильтрацию по метаданным (hybrid: "фильтр + ANN");
- управление коллекциями, репликацию/шардинг, обновления, часто — API для retrieval в прикладных системах (например, RAG).

Примеры (популярные семейства решений):

- специализированные vector DB: **Milvus**, **Qdrant**, **Weaviate**, **Pinecone**, **Chroma**, **LanceDB**
- расширения к классическим хранилищам: **PostgreSQL + pgvector**, **Elasticsearch/OpenSearch**, **Redis (Vector/RediSearch)**;
- библиотеки/движки ANN (не БД, но часто лежат внутри): **FAISS**, **ScaNN**, реализации **HNSW**.

19 Метрики классификации

19.1 Определение метрики в ML

19.1.1 Свойства, сравнение с функцией потерь

Метрика – численная функция качества модели на validation/test. Свойства: интерпретируемость, согласованность с задачей, стабильность, сопоставимость.
Loss vs метрика:

- **Loss:** оптимизируется при обучении, обычно гладкая/выпуклая.
- **Метрика:** оценивает качество, может быть недифференцируемой (accuracy, F1).

Часто минимизируют surrogate-loss (log-loss), а измеряют другой метрикой (ROC-AUC, F1).

19.2 Матрица ошибок (бинарная, многоклассовая)

19.2.1 Типы ошибок

19.2.2 Правило присвоения меток

Матрица ошибок — таблица распределения предсказаний по истинным классам.
Бинарный случай: TP (true positive), TN (true negative), FP (false positive, ошибка I рода), FN (false negative, ошибка II рода).
Многоклассовый: матрица $K \times K$, диагональ – верные ответы.
Правило присвоения: если модель выдаёт $\hat{p}(x)$, то $\hat{y} = \mathbb{I}[\hat{p}(x) \geq \tau]$. Порог τ подбирают по метрике (F1, recall@precision).

19.3 Accuracy, balanced accuracy, precision, recall, F1-score

Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$ (доля верных ответов, для сбалансированных классов).
BalancedAcc = $\frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right)$ (полезна при дисбалансе).
Precision = $\frac{TP}{TP+FP}$ ("если модель скажала 1, как часто это правда").
Recall = $\frac{TP}{TP+FN}$ ("какую долю положительных мы нашли").
$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP+FP+FN}$ (компромисс между precision и recall).

19.3.1 Sensitivity, specificity и другие

Sensitivity = TPR = Recall = $\frac{TP}{TP+FN}$, **Specificity** = TNR = $\frac{TN}{TN+FP}$.
FPR = $\frac{FP}{FP+TN}$ = 1 − Specificity, FNR = $\frac{FN}{FN+TP}$ = 1 − Sensitivity.
NPV = $\frac{TN}{TN+FN}$ (negative predictive value), FDR = $\frac{FP}{FP+TP}$ = 1 − Precision (false discovery rate).

19.3.2 Несбалансированная стоимость ошибок, примеры, как исправлять дисбаланс

Стоимость FP и FN различается: медицина (FN дорожe), антифрод (FN дорог, но много FP — неудобства), безопасность (FN критичен).
Как исправлять:

- Подбор порога τ по precision/recall.
- Взвешивание классов в loss (class weights).
- Ресэмплинг (undersampling/oversampling, SMOTE).
- Подбор метрики (balanced accuracy, F1, recall@precision).

21 Метод опорных векторов (SVM)

21.1 Геометрическая интерпретация

Бинарная классификация $y_i \in \{-1, +1\}$, скор $f(x) = w^\top x + b$. Разделяющая гиперплоскость: $w^\top x + b = 0$.
Линейно разделимый случай: SVM максимизирует зазор (margin) между классами. Опорные гиперплоскости $w^\top x + b = \pm 1$. Условие: $y_i(w^\top x_i + b) \geq 1$. Ширина полосы: $\text{margin} = \frac{2}{\|w\|_2} \to$ минимизируем $\|w\|_2^2$.
Опорные векторы: объекты на границах полосы, определяющие гиперплоскость.

21.2 Оптимизационная задача для линейно разделимого случая

21.2.1 * Двойственная задача

Прямая задача (hard-margin): $\min_{w,b} \frac{1}{2}\|w\|_2^2$ s.t. $y_i(w^\top x_i + b) \geq 1$.
Лагранжиан: $\mathcal{L} = \frac{1}{2}\|w\|_2^2 - \sum \alpha_i (y_i(w^\top x_i + b) - 1)$. KKT: $w = \sum \alpha_i y_i x_i$, $\sum \alpha_i y_i = 0$.
Двойственная задача: $\max_\alpha \sum \alpha_i - \frac{1}{2}\sum \alpha_i \alpha_j y_i y_j (x_i^\top x_j)$ s.t. $\alpha_i \geq 0$, $\sum \alpha_i y_i = 0$.
Восстановление: $w^* = \sum \alpha_i^* y_i x_i$, $b^* = y_s - w^{*\top} x_s$ (по опорному вектору x_s с $\alpha_s^* > 0$).

21.3 Неразделимый случай, slack-переменные

Soft-margin SVM: $\min_{w,b,\xi} \frac{1}{2}\|w\|_2^2 + C\sum \xi_i$ s.t. $y_i(w^\top x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$.
$C > 0$ – компромисс: большое $C \to$ подгонка под train, малое $C \to$ шире полоса (регуляризация).
Геометрический смысл ξ_i. Если $\xi_i = 0$, объект лежит на правильной стороне полосы: $y_i(w^\top x_i + b) \geq 1$. Если $0 < \xi_i < 1$, объект классифицирован верно, но находится внутри полосы. Если $\xi_i \geq 1$, объект либо на границе, либо классифицирован неверно (зажка неположительна).

21.4 Hinge loss

Обозначим **положительную часть** (positive part):

$$[\cdot]_+ = \max(0, \cdot).$$

Тогда **hinge loss** для объекта (x_i, y_i):

$$\ell_{\text{hinge}}(y_i, f(x_i)) = [1 - y_i f(x_i)]_+ = \max(0, 1 - y_i(w^\top x_i + b)).$$

Интерпретация:

- если $y_i(w^\top x_i + b) \geq 1$, то $\ell_{\text{hinge}} = 0$ (объект вне полосы, запас маржи достаточен);
- если $y_i(w^\top x_i + b) < 1$, то штраф линейно растёт по мере уменьшения маржи.

Связь hinge loss и soft-margin. Из ограничений $y_i(w^\top x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$ следует, что минимально допустимое ξ_i равно

$$\xi_i^* = [1 - y_i(w^\top x_i + b)]_+.$$

Подставляя это в целевую функцию, получаем эквивалентную (без явных ξ) форму:

$$\min_{w,b} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1} [1 - y_i(w^\top x_i + b)]_+.$$

То есть hinge loss "извлекается" из ограничений soft-margin постановки и дополняется L_2-регуляризацией, отвечающей за ширину полосы.

21.5 Ядровой трюк, свойства ядер, примеры ядер

Линейный SVM строит гиперплоскость в исходном пространстве признаков. Для нелинейной разделимости применяют **переход в признаковое пространство**:

$$\varphi : \mathbb{R}^d \to \mathcal{H}, \qquad x \mapsto \varphi(x),$$

где $\mathcal{H}$ — (возможно, высоко- или бесконечномерное) гильбертово пространство. В $\mathcal{H}$ модель остаётся линейной:

$$f(x) = \langle w, \varphi(x) \rangle_{\mathcal{H}} + b.$$

23 k ближайших соседей (kNN) для классификации и регрессии

23.1 Параметры и гиперпараметры

kNN (*k Nearest Neighbors*) — **непараметрическая** модель: в ней **нет обучаемых параметров** вида w, b, а "обучение" по сути сводится к **сохранению обучающей выборки** (возможно с предварительной индексацией). Все предсказания строятся по ближайшим объектам.
Базовый алгоритм предсказания. Пусть дана обучающая выборка $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ и метрика (функция расстояния) $d(\cdot, \cdot)$. Для нового объекта x:

1. вычисляем расстояния $d(x, x_i)$ до всех (или до кандидатов из индекса) объектов обучающей выборки;
2. выбираем множество индексов $N_k(x)$ – k ближайших соседей:
$$N_k(x) \in \arg\min_{S \subseteq \{1,\dots,n\}, |S|=k} \sum_{i \in S} d(x, x_i);$$
3. агрегируем ответы $\{y_i\}_{i \in N_k(x)}$.

Агрегация для классификации. В простейшем варианте используется **голосование большинством**:

$$\hat{y}(x) = \arg\max_{c \in \mathcal{C}} \sum_{i \in N_k(x)} \mathbb{I}\{y_i = c\},$$

где $\mathcal{C}$ — множество классов.
Иногда используют **взвешенное голосование**:

$$\hat{y}(x) = \arg\max_{c \in \mathcal{C}} \sum_{i \in N_k(x)} w_i(x)\, \mathbb{I}\{y_i = c\},$$

где веса $w_i(x)$ могут зависеть от *порядка соседа* или от *самого расстояния* (например, $w_i(x) = 1/(d(x, x_i) + \varepsilon)$).
Агрегация для регрессии. Типичный ответ – **среднее** по соседям:

$$\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i,$$

или **взвешенное среднее**:

$$\hat{y}(x) = \frac{\sum_{i \in N_k(x)} w_i(x)\, y_i}{\sum_{i \in N_k(x)} w_i(x)}.$$

(Аналогично можно использовать медиану, если нужно повысить устойчивость к выбросам в y.)
Параметры и гиперпараметры kNN.

- **Параметры (trainable parameters): отсутствуют.** Это соответствует тому, что kNN – **non-parametric model**: значения параметров не находятся решением задачи оптимизации
- **Гиперпараметры (задаются до применения модели):**
  1. k **— число соседей.** Малые k делают модель более "локальной" (обычно выше вариативность ответов), большие k — более "гладкой" (обычно сильнее усреднение и выше смещение). Частный случай $k = 1$ даёт максимально локальное решение.
  2. **Функция расстояния** $d(\cdot, \cdot)$. На практике выбор метрики критичен, так как именно она определяет понятие "близости":
     - Евклидово расстояние (L_2): $d(x, z) = \|x - z\|_2$;
     - Манхэттенское расстояние (L_1): $d(x, z) = \|x - z\|_1$;
     - расстояния Минковского: $d(x, z) = \|x - z\|_p$ (гиперпараметр p);
     - косинусная "дистанция" (через косинусное сходство);
     - Хэммингово расстояние (число несовпадений/замен) и др.
  3. **Схема взвешивания соседей** (используется существенно реже базового голосования, веса могут зависеть от ранга соседа или от расстояния до соседа.

25 The bitter lesson (Горький урок)

25.1 Основные выводы статьи

The Bitter Lesson — эссе Рича Саттона (2019), где он формулирует исторически повторяющийся вывод из десятилетий развития ИИ: в долгосрочной перспективе **побеждают общие методы**, которые **масштабируются по вычислениям и данным**, а не подходы, основанные на тщательном **встраивании человеческих знаний о предметной области**.

1. **Главный тезис: "масштабируемые общие методы" в итоге выигрывают.**
   Во многих направлениях ИИ наблюдался один и тот же паттерн: сначала доминируют решения, насыщенные человеческой экспертизой (правила, эвристики, ручные признаки, специальные структуры), но затем их обгоняют более общие подходы, которые лучше используют рост вычислительных мощностей и объёмов данных.
2. **Почему "урок горький": антропоцентрические решения психологически привлекательны, но стратегически проигрывают.**
   Инженеру и исследователю приятно строить систему "как человек думает":
   - что даёт ощутимый **быстрый прогресс** на малых масштабах;
   - система может касаться **интерпретируемой** и "разумной";
   - но такие решения часто становятся **потолком**: добавление новых эвристик усложняет систему, делает её хрупкой и не позволяет эффективно конвертировать рост compute/data в рост качества.
3. **Критерий жизнеспособности метода: он обязан улучшаться при росте compute и data.**
   Саттон подчёркивает, что методы, которые:
   - **получают выгоду от большего числа вычислений** (например, более длинный поиск, больше шагов оптимизации, более крупные модели, более длинное обучение),
   - **получают выгоду от большего объёма данных** (обучение представлений, обобщен на новых примерах),

   имеют систематическое преимущество: их качество растёт естественным образом по мере развития железа и накопления данных, тогда как "ручные" улучшения имеют убывающую отдачу и быстро устаревают.
4. **Два класса общих методов, которые исторически лучше всего масштабируются: search и learning.**
   В терминах эссе:
   - **Search (поиск):** систематический перебор/планирование/оптимизация, где больше вычислений ⇒ глубже/шире поиск ⇒ лучшее решение.
   - **Learning (обучение):** методы, которые автоматически извлекают закономерности из данных (например, градиентное обучение параметрических моделей), где больше данных и шагов обучения ⇒ лучшее качество.

   Важно, что эти подходы не завязаны на конкретную предметную область и потому повторно применимы.
5. **Исторические примеры (как иллюстрация повторяемости урока).**
   В разных доменах (игры, речь, зрение и др.) итоговый скачок качества обычно достигался не за счёт усложнения доменных правил, а за счёт методов, которые лучше масштабируются
   - переход от "умных" ручных эвристик к методам, активно использующим вычисления (поиск/оптимизацию);
   - переход от ручных признаков к обучаемым представлениям на больших датасетах;
   - рост роли аппаратного ускорения и параллельных вычислений.
6. **Практический вывод для ML-инженера: проектируй решения так, чтобы они масштабировались.**
   При выборе архитектуры/алгоритма и постановке экспериментов полезно задавать вопросы
   - Что произойдёт с качеством, если увеличить датасет в 2–10 раз?
   - Что произойдёт, если увеличить бюджет обучения (эпохи, параметры модели, ширину поиска)?
   - Не ограничивает ли решение заранее "зашитая" структура (эвристики/правила/ручные фичи) способность модели учиться?
   - Можно ли автоматизировать извлечение признаков и улучшение поведения через обучение?
7. **Баланс: доменные знания полезны, но их роль должна быть "поддерживающей", а не "несущей".**
   Доменные знания оправданы как:
   - инженерные ограничения (безопасность, латентность, стоимость),
   - корректная постановка задачи, метрики, протокол валидации,
   - разумные индуктивные смещения, которые **не ломают масштабирование**.

   Но ставка на "ручную имитацию мышления" как основной источник прогресса чаще всего ведёт к тупику.

19.4 ROC AUC

19.4.1 Свойства ROC и ROC-AUC

ROC-кривая – график TPR(τ) от FPR(τ) при переборе порога. **ROC-AUC** – площадь под ROC: ROC-AUC = $\int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR})$.
Свойства:

- Интерпретация: вероятность, что случайный positive получит скор выше случайного negative.
- Не зависит от порога (оценивает ранжирующую способность).
- Менее чувствителен к дисбалансу, чем accuracy, но может быть оптимистичным при сильном дисбалансе.

19.4.2 PR AUC

PR-кривая: зависимость precision от recall при переборе τ. **PR-AUC** – площадь под PR-кривой.
Полезна при сильном дисбалансе и когда важен контроль FP (антифрод, поиск редких событий).
Информативнее ROC-AUC в таких задачах.

19.5 Многоклассовые расширения

19.5.1 Micro- и macro-усреднение

Для K классов определяем TP_k, FP_k, FN_k для каждого класса k против остальных.
Macro: усреднение метрики по классам (все классы равновесны): $\text{Precision}_{\text{macro}} = \frac{1}{K}\sum_k \text{Precision}_k$.
Важно качество на каждом классе.
Micro: суммирование TP/FP/FN по классам: $TP_{\text{micro}} = \sum_k TP_k$, затем $\text{Precision}_{\text{micro}} = \frac{TP_{\text{sum}}}{TP_{\text{sum}}+FP_{\text{sum}}}$.
Отражает качество на частых классах.

20 Матричные нормы

20.1 p-нормы, самые важные частные случаи

Нормы используются для измерения "размера" векторов и матриц, а в ML особенно важны для регуляризации (L_1/L_2), оценки устойчивости и анализа обусловленности.
1) p-норма вектора. Для $x \in \mathbb{R}^d$ и $p \geq 1$:

$$\|x\|_p = \left(\sum_{i=1}^d |x_i|^p\right)^{1/p}.$$

Также часто используют предельную норму:

$$\|x\|_\infty = \max_{1 \leq j \leq d} |x_j|.$$

Важные частные случаи (векторные нормы):

$$\|x\|_1 = \sum_{j=1}^d |x_j| \quad \text{(L1, разреженная регуляризация)},$$

$$\|x\|_2 = \left(\sum_{j=1}^d x_j^2\right)^{1/2} \quad \text{(L2, "евклидова")},$$

$$\|x\|_\infty = \max_j |x_j| \quad \text{(контроль максимального отклонения)}.$$

2) Индуцированная (операторная) p-норма матрицы. Для матрицы $A \in \mathbb{R}^{m \times d}$ индуцированная норма определяется через действие на вектора:

$$\|A\|_p = \max_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p} = \max_{\|x\|_p = 1} \|Ax\|_p.$$

Эта норма естественно измеряет, насколько сильно линейное преобразование $x \mapsto Ax$ может "растянуть" вектор в соответствующей p-норме.
Важные частные случаи (индуцированные матричные нормы):

- $p = 1$**: максимум суммы по столбцу.**
$$\|A\|_1 = \max_{1 \leq j \leq d} \sum_{i=1}^m |a_{ij}|.$$
- $p = \infty$**: максимум суммы по строке.**
$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^d |a_{ij}|.$$
- $p = 2$**: спектральная норма.**
$$\|A\|_2 = \max_{\|x\|_2 = 1} \|Ax\|_2 = \sqrt{\lambda_{\max}(A^\top A)}.$$
Здесь $\lambda_{\max}(A^\top A)$ – максимальное собственное значение матрицы $A^\top A$.

3) Норма Фробениуса (часто используется в ML). Хотя она не является индуцированной p-нормой, это одна из наиболее популярных матричных норм:

$$\|A\|_F = \left(\sum_{i=1}^m \sum_{j=1}^d a_{ij}^2\right)^{1/2} = \sqrt{\text{tr}(A^\top A)}.$$

Интерпретация: это евклидова норма вектора, полученного "разворачиванием" матрицы в один столбец.
Ключевые свойства (в контексте ML):

- **Монотонность по p для вектора:** при $1 \leq p \leq q \leq \infty$ верно $\|x\|_q \leq \|x\|_p$ (при фиксированном x).
- **Субмультипликативность индуцированных норм:**
$$\|AB\|_p \leq \|A\|_p \|B\|_p,$$
что важно для анализа устойчивости композиции линейных преобразований.

22 Метод главных компонент (PCA)

22.1 Связь с SVD

Пусть дана матрица данных $X \in \mathbb{R}^{n \times d}$ (строки — объекты, столбцы — признаки). PCA предполагает **центровку**: $X_c = X - \mathbf{1}\mu^\top$, где $\mu = \frac{1}{n}\sum_{i=1}^n x_i$.
SVD разложение: $X_c = U\Sigma V^\top$, где $U \in \mathbb{R}^{n \times r}$, $V \in \mathbb{R}^{d \times r}$ ортонормированы, $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r$, $\sigma_1 \geq \dots > \sigma_r > 0$.
Связь PCA и SVD:

- **Главные направления** — столбцы V (правые сингулярные векторы).
- **Проекции (scores)**: $Z = X_c V = U\Sigma \in \mathbb{R}^{n \times r}$.
- **Ковариационная матрица**: $C = \frac{1}{n-1}X_c^\top X_c = \frac{1}{n-1}V\Sigma^2 V^\top$, где $\lambda_j = \frac{\sigma_j^2}{n-1}$ — собственные значения.
- **Объяснённая дисперсия**: $\text{EVR}(k) = \frac{\sum_{j=1}^k \sigma_j^2}{\sum_{j=1}^r \sigma_j^2}$.

Снижение размерности. Берём первые k компонент: $V_k = [v_1, \dots, v_k] \in \mathbb{R}^{d \times k}$, $Z_k = X_c V_k$.
Реконструкция: $\hat{X}_c = Z_k V_k^\top = X_c V_k V_k^\top$.

22.2 Теорема Эккарта–Юнга, доказательство

22.2.1 способ через собственное разложение или через подстановку

Теорема Эккарта–Юнга–Мирски. Для $A = U\Sigma V^\top$ лучшая аппроксимация ранга k в норме Фробениуса:

$$A_k = U_r \Sigma_k V_r^\top \in \arg\min_{\text{rank}(B) \leq k} \|A - B\|_F, \qquad \min_{\text{rank}(B) \leq k} \|A - B\|_F^2 = \sum_{j=k+1}^r \sigma_j^2.$$

Доказательство (через подстановку). Используя ортогональную инвариантность $\|Q_1 M Q_2\|_F = \|M\|_F$, для $C = U^\top BV$ получаем:

$$\min_{\text{rank}(B) \leq k} \|A - B\|_F = \min_{\text{rank}(C) \leq k} \|\Sigma - C\|_F.$$

Оптимально брать C диагональной с максимальными k элементами: $C^* = \text{diag}(\sigma_1, \dots, \sigma_k, 0, \dots,$ откуда $B^* = U_k \Sigma_k V_k^\top$.
Следствие для PCA. Реконструкция $X_c V_k V_k^\top$ минимизирует ошибку восстановления среди всех k-мерных подпространств.

22.3 Две альтернативные итеративные формулировки

(1) Жадный выбор направления максимальной дисперсии. Для ковариационной матрицы $C = \frac{1}{n-1}X_c^\top X_c$ первая компонента: $v_1 = \arg\max_{\|v\|_2=1} v^\top Cv$ — собственный вектор с максимальным λ_1. Далее выбирают $v_2 \perp v_1$ с максимальной дисперсией и т.д.
(2) Удаление направления минимальной дисперсии. PCA удаляет направления с наименьшим λ_j, оставляя k направлений с максимальной дисперсией.

Выбор k: часто по порогу объяснённой дисперсии $\frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^r \lambda_j} \geq \tau$ (например, $\tau = 0.9$).

22.4 Предобработка для PCA

PCA чувствителен к масштабу признаков.
1) Центровка (обязательно): $X_c = X - \mathbf{1}\mu^\top$.
2) Нормализация (обычно необходима):

- **StandardScaler:** $x_{ij}^{\text{std}} = \frac{x_{ij} - \mu_j}{s_j}$.
- **MinMaxScaler:** $x_{ij}^{\text{mm}} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}$.

24 Индексы для kNN

24.1 Зачем нужны индексы

В kNN поиск k ближайших точек наивно требует $O(nd)$ времени. **Индексы** нужны для: ускорения инференса, снижения вычислительных затрат, работы с high-dimensional эмбеддингами ($d \sim 128$–4096, и до миллионов), управления trade-off точность–скорость.
Проклятие размерности: в высоких d дерево-индексы деградируют, используют ANN (графы и т.п.).

24.2 Идеи индексов для kNN от простых к сложным

1. **Брутфорс + оптимизации:** SIMD, батчинг, BLAS, нормализация для косинусной близости
2. **Фильтры:** двухступенчатая схема retrieval → rerank.
3. **Точные геометрические (малые d):** kd-tree (по координатам), ball-tree (по радиусам).
4. **Хеширование/проекции (ANN):** LSH (близкие → одинаковые бакеты), random projection trees.
5. **Графовые ANN:** HNSW (иерархический граф малых миров) — state-of-the-practice.
6. **Квантование (большие n):** PQ, IVF+PQ (сжатие+кластеризация).

24.3 k-dimensional tree (kd-tree)

kd-tree – бинарное дерево разбиений по осям координат. Эффективен в низкой d (до десятков признаков).
Построение: выбор оси j (циклически/по макс. дисперсии) и порога t (медиана), разбиение $\{x : x_j \leq t\}$ и $\{x : x_j > t\}$. Глубина $O(\log n)$, построение $O(n \log n)$.
Поиск: спуск по дереву, инициализация радиуса r, backtracking (посещаем альтернативную ветвь если $|x_j - t| < r$).
Плюсы: точный поиск, хорошо при малом d. **Минусы:** деградация при росте d.

24.4 Annoy

Annoy — ANN на лесах деревьев случайных проекций. Каждое дерево рекурсивно режет пространство случайными гиперплоскостями.
Поиск: проход по деревьям до листьев, объединение кандидатов, top-k по точной метрике.
Гиперпараметры: число деревьев T (больше ⇒ лучше recall), глубина поиска (больше ⇒ выше latency).
Особенности: для статических коллекций, косинусная/евклидова метрика, retrieval → rerank.

24.5 HNSW

HNSW (*Hierarchical Navigable Small World*) — графовый ANN-индекс. Он хранит **многослойный** граф: на верхних слоях мало вершин и длинные "прыжки", на нижних слоях много вершин и локальные связи. Это обеспечивает быстрый навигационный поиск.

24.5.1 Построение графа

Пусть есть точки $\{x_i\}_{i=1}^n$ и метрика/дистанция $d(\cdot, \cdot)$. Каждая точка получает **слой** (уровень) $L(i) \in \{0, 1, 2, \dots\}$, где слой 0 — самый плотный. Обычно уровни распределяют так, чтобы большинство точек были на нижнем уровне, а на верхних – всё меньше.
Вставка новой вершины x (общая схема):

1. **Выбор уровня.** Сэмплируется максимальный уровень $L(x)$ (вершина присутствует на уровнях $0, \dots, L(x)$).
2. **Поиск точки входа.** Начиная с текущей точки входа (*entry point*) на верхнем уровне, выполняют **жадный поиск** (greedy): переходят к соседу, который ближе к x, пока улучшений нет.

26 Деревья решений

26.1 Определение жадных алгоритмов

Жадный алгоритм – выбор локально оптимального решения на каждом шаге. Не гарантирует глобальный оптимум, но дает разумное приближение.

26.2 Процедура построения

Дерево строится рекурсивно. **CART**: для признака j и порога t разбиение $L(j,t) = \{x_{ij} \leq t\}$, $R(j,t) = \{x_{ij} > t\}$. Жадный шаг: выбор (j^*, t^*) максимизирующего критерий. **Остановка:** макс. глубина, мало объектов, чистый узел. Поиск через перебор порогов по отсортированным значениям. Деревья слабо чувствительны к нормировке.

26.3 Информационные критерии, энтропия, индекс Джини

Качество: $\Delta H(j,t) = H(Q) - \frac{n_l}{n}H(L) - \frac{n_r}{n}H(R)$. Выбор: $(j^*, t^*) = \arg\max \Delta H$.
Классификация ($p_k(S)$ — доля класса k): Misclassification $H_{\text{mis}} = 1 - \max_k p_k$. Энтропия $H_{\text{ent}} = -\sum_k p_k \log p_k$. Джини $H_{\text{gini}} = 1 - \sum_k p_k^2$.
Регрессия: MSE $H_{\text{MSE}} = \frac{1}{|S|}\sum (y_i - \bar{y}_S)^2$ или MAE.

26.4 Ответ в листе

Классификация: самый частый класс $\hat{y} = \arg\max_k p_k(S)$ или распределение $\hat{p}_k = p_k(S)$.
Регрессия: MSE → среднее $\hat{y} = \bar{y}_S$; MAE → медиана (робастнее).

26.5 Обработка пропусков

Стратегии для NaN: импутация до обучения, отдельная ветка для NaN, C4.5 (распределить по обеим веткам), CART (surrogate splits – альтернативные разбиения).

26.6 Бинаризация

Заранее задать пороги $t_{j,1} < \dots < t_{j,B}$, рассматривать $\phi_{jb}(x) = \mathbb{I}[x_j \leq t_{jb}]$. Уменьшает сложность выбора порога.

26.7 Обрезка (pruning), варианты C4.5 и CART

C4.5 (pessimistic): оценка верхней границы ошибки через доверительный интервал, замена поддерева листом если выгодно.
CART (cost-complexity): $R_\alpha(T) = R(T) + \alpha|T|$, последовательность деревьев при возрастающем α, выбор по validation.

26.8 Гиперпараметры (минимум 4 основных)

max_depth (глубина), **min_samples_split** (мин. объектов для разбиения), **min_samples_leaf** (мин. в листе), **min_impurity_decrease** (мин. улучшение), **criterion** (gini/entropy/MSE/MAE), **max_features** (число признаков).

Ядровой трюк (kernel trick). В двойственной задаче и в предсказании SVM данные входят только через скалярные произведения $x_i^\top x_j$. Если заменить их на

$$K(x_i, x_j) = \langle \varphi(x_i), \varphi(x_j) \rangle_{\mathcal{H}}$$

то можно обучать и применять SVM в $\mathcal{H}$, **не вычисляя** $\varphi(x)$ явно.
Dual-форма с ядром. Двойственная задача (soft/hard margin различаются ограничениями на α, но структура одинакова) принимает вид:

$$\max_\alpha \sum_{i=1}^n \alpha_i - \frac{1}{2}\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0, \;\; \alpha_i \geq 0.$$

В soft-margin добавляется верхняя граница $\alpha_i \leq C$.
Решающее правило (decision function). Итоговая функция классификации выражается через опорные векторы:

$$f(x) = \sum_{i=1}^n \alpha_i^* y_i K(x_i, x) + b^*, \qquad \hat{y} = \text{sign}(f(x)).$$

Свойства ядра. Функция $K(\cdot, \cdot)$ является допустимым ядром, если:

- **симметрична:** $K(x, z) = K(z, x)$;
- **положительно полуопределена (PSD):** для любых точек $x_1, \dots, x_n$ матрица Грама $[K(x_i, x_j)]_{i,j=1}^n$ является PSD, то есть для любого $c \in \mathbb{R}^n$
$$c^\top Kc \geq 0.$$

Практически это означает, что K соответствует некоторому скалярному произведению в некотором гильбертовом пространстве. Также полезны замкнутости: суммы, масштабирование и некоторые композиции допустимых ядер снова дают ядра.
Примеры ядер (часто используемые):

- **Линейное ядро:**
$$K(x, z) = x^\top z.$$
- **Полиномиальное ядро:**
$$K(x, z) = (\gamma x^\top z + r)^d, \qquad \gamma > 0,\; r \geq 0,\; d \in \mathbb{N}.$$
- **Гауссовское RBF-ядро:**
$$K(x, z) = \exp\{-\gamma \|x - z\|_2^2\}, \qquad \gamma > 0.$$
- **Экспоненциальное (Laplacian) ядро:**
$$K(x, z) = \exp\{-\gamma \|x - z\|_2\}, \qquad \gamma > 0.$$

Замечание про "гиперпараметры реализации" (скорость/память). В прикладных системах для ускорения поиска ближайших соседей часто вводят индексные структуры и приближённый поиск (ANN), и тогда добавляются настройки, влияющие прежде всего на вычислительную эффективность (например, выбор структуры индекса и её параметры), но не статистическая постановка kNN при этом остаётся той же: определить $N_k(x)$ по выбранной метрике и агрегировать ответы.

27 Разложение bias–variance–noise

27.1 Случайность в ошибке

27.1.1 Интуиция каждой компоненты

Постановка: $Y = f(X) + \varepsilon$, $\mathbb{E}[\varepsilon \mid X] = 0$, $\mathrm{Var}(\varepsilon \mid X) = \sigma^2(X)$. Алгоритм на $\mathcal{D}$ даёт $\hat{f}_{\mathcal{D}}(\cdot)$.
Источники случайности: инициализация, SGD, сэмплирование $\mathcal{D}$, шум ε, разметка.
Центральная модель: $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$.
Определения:

$$\mathrm{Bias}(x) = \bar{f}(x) - f(x), \quad \mathrm{Var}(x) = \mathbb{E}_{\mathcal{D}}\big[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2\big], \quad \mathrm{Noise}(x) = \sigma^2(x).$$

Интуиция:

- **Bias** — систематическая ошибка (underfitting).
- **Variance** — чувствительность к $\mathcal{D}$ (overfitting).
- **Noise** — неустранимая ошибка в Y.

27.2 Доказательство для MSE

Для фиксированного x, подставим $Y = f(x) + \varepsilon$:

$$\mathbb{E}_{\mathcal{D},\varepsilon}\big[(Y - \hat{f}_{\mathcal{D}}(x))^2\big] = \mathbb{E}_{\mathcal{D}}\big[(f(x) - \hat{f}_{\mathcal{D}}(x))^2\big] + \sigma^2(x),$$

где перекрёстный член $\mathbb{E}[\varepsilon(f(x) - \hat{f}_{\mathcal{D}}(x))] = 0$.
Разложение первого члена:

$$\mathbb{E}_{\mathcal{D}}\big[(\hat{f}(x) - \bar{f}_{\mathcal{D}}(x))^2\big] = \underbrace{\mathbb{E}_{\mathcal{D}}\big[(\bar{f}_{\mathcal{D}}(x) - \bar{f}(x))^2\big]}_{\mathrm{Var}(x)} + \underbrace{(\bar{f}(x) - f(x))^2}_{\mathrm{Bias}(x)^2}.$$

Итого:

$$\boxed{\mathbb{E}_{\mathcal{D},\varepsilon}\big[(Y - \hat{f}_{\mathcal{D}}(x))^2 \mid X = x\big] = \mathrm{Bias}(x)^2 + \mathrm{Var}(x) + \sigma^2(x).}$$

27.3 Компромисс bias–variance

27.3.1 Анализ основных моделей

Компромисс: при росте сложности модели bias ↓, variance ↑ (U-образная кривая ошибки).
Базовые модели:

Модель	Bias	Variance
Линейные	Высокий (если нелинейная зависимость)	Низкий
kNN	Малый k: низкий; большой k: высокий	Малый k: высокий; большой k: низкий
Деревья	Глубокие: низкий; мелкие: высокий	Глубокие: высокий
Bagging/RF	Близок к базовой модели	Существенно снижается
Бустинг	Сильно снижается	Контролируется регуляризацией
Нейросети	При достаточной параметризации низкий	Зависит от регуляризации/данных

Управление: Ridge/Lasso $\lambda \uparrow$, kNN $k \uparrow$, деревья max_depth↓ ⇒ bias↑, variance↓.

27.4 За пределами MSE

Для MSE: Risk = Bias^2 + Var + Noise.
Для других лоссов (0–1, log-loss) разложение сложнее. Обобщённые подходы: $\mathbb{E}[\mathrm{loss}]$ = loss of cen variation term.

27.5 Double descent

Double descent — современное явление: тестовая ошибка(ёмкость) имеет не U-, а W-форму.
Схема:

1. **Первый спуск:** рост ёмкости ⇒ bias↓, ошибка↓.
2. **Пик (interpolation threshold):** модель едва подгоняет train ⇒ ошибка↑ (подгонка шума).
3. **Второй спуск (overparameterized):** дальнейший рост ёмкости ⇒ ошибка↓ (неявная регуляризация).

Вывод: очень большие модели могут работать лучше, чем модели на границе интерполяции.

29.2 One-Class SVM

One-Class SVM — метод обнаружения новизны/аномалий, который строит границу, отделяющую "нормальные" объекты от "всего остального" в пространстве признаков (часто в **ядровом** пространстве).
Интуиция. Мы хотим найти поверхность (гиперплоскость в $\mathcal{H}$), которая:

- включает большую часть обучающих точек (норму);
- допускает небольшую долю выбросов;
- имеет максимально возможный зазор (регуляризация через $\|w\|_2$).

Постановка (primal). Пусть есть только "нормальные" примеры $\{x_i\}_{i=1}^n$ и отображение $\varphi(x)$ в признаковое пространство $\mathcal{H}$. Оптимизационная задача one-class SVM:

$$\min_{w,\rho,\xi} \frac{1}{2}\|w\|^2 + \frac{1}{\nu n}\sum_{i=1}^n \xi_i - \rho$$
$$\text{s.t.} \quad \langle w, \varphi(x_i)\rangle \geq \rho - \xi_i, \quad i = 1,\dots,n, \qquad \xi_i \geq 0,$$

где:

- w задаёт гиперплоскость в $\mathcal{H}$,
- ρ – смещение (порог),
- ξ_i – slack-переменные (разрешают нарушения),
- $\nu \in (0, 1]$ – гиперпараметр, управляющий долей выбросов/опорных векторов.

Смысл параметра ν. В классической интерпретации:

- ν является **верхней оценкой** доли объектов, которые могут оказаться выбросами (ошибка
- ν является **нижней оценкой** доли опорных векторов.

Двойственная задача (dual) и ядровой трюк. В двойственной форме появляются коэффици α_i и ядро $K(x, z) = \langle\varphi(x), \varphi(z)\rangle$:

$$\max_{\alpha} -\frac{1}{2}\sum_{i=1}^n\sum_{j=1}^n \alpha_i\alpha_j K(x_i, x_j)$$
$$\text{s.t.} \quad 0 \leq \alpha_i \leq \frac{1}{\nu n}, \quad i = 1,\dots,n, \qquad \sum_{i=1}^n \alpha_i = 1.$$

Решающее правило:

$$f(x) = \sum_{i=1}^n \alpha_i K(x_i, x) - \rho, \qquad \hat{y}(x) = \mathrm{sign}(f(x)).$$

Обычно интерпретируют так:
$\hat{y}(x) = +1 \Rightarrow$ "норма", $\hat{y}(x) = -1 \Rightarrow$ "аномалия/новизна".
Ядра и гиперпараметры. One-Class SVM часто применяют с RBF-ядром:

$$K(x, z) = \exp\big(-\gamma\|x - z\|_2^2\big), \qquad \gamma > 0.$$

Тогда основные гиперпараметры:

- ν — доля допускаемых выбросов / жёсткость границы;
- γ (или эквивалентно ширина RBF) — локальность границы;
- выбор ядра (linear / polynomial / RBF) и его параметры;
- предобработка: центровка/масштабирование признаков (для RBF обычно критично).

Сравнение с Isolation Forest (высокоуровнево).

- One-Class SVM часто даёт сильную границу на умеренных n, но может быть тяжёлым по времени/памяти на больших датасетах (ядровые методы масштабируются хуже).
- Isolation Forest обычно лучше масштабируется на большие n и проще в эксплуатации как быстрый unsupervised baseline.

28.4 Подход SHAP (Shapley additive explanations)

28.4.1 Аксиомы, формула

SHAP основан на **значениях Шепли** из теории игр: признаки — "игроки", ценность коалиции $v(S)$.
Аксиомы:

- **Efficiency:** $\sum_{j=1}^d \phi_j = v(\{1,\dots,d\}) - v(\varnothing)$.
- **Symmetry:** одинаковый вклад ⇒ равные объяснения.
- **Dummy player:** нулевой вклад ⇒ $\phi_j = 0$.
- **Additivity:** для суммы игр значения суммируются.

Формула Шепли для признака j:

$$\boxed{\phi_j(v) = \sum_{S\subseteq N\setminus\{j\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!}\Big(v(S\cup\{j\}) - v(S)\Big).}$$

28.4.2 SHAP-значения для ML

Для ML с предикторами $f : \mathbb{R}^d \to \mathbb{R}$ и объекта x определяем ценность коалиции:

$$v_x(S) = \mathbb{E}\big[f(X) \mid X_S = x_S\big], \qquad v_x(\varnothing) = \mathbb{E}[f(X)].$$

Аддитивное объяснение:

$$\boxed{f(x) = \phi_0 + \sum_{j=1}^d \phi_j(x), \qquad \phi_0 = \mathbb{E}[f(X)].}$$

$\phi_j(x) > 0$ — признак увеличивает предсказание; $\phi_j(x) < 0$ — уменьшает.
Инсайт: при коррелированных признаках разные приближения дают разные SHAP-значения.

28.4.3 KernelSHAP, TreeSHAP

KernelSHAP — model-agnostic приближение через взвешенную регрессию на бинарных масках. Рассматриваем маски $z \in \{0,1\}^d$ ($z_j = 1$ — признак доступен), обучаем линейную модель $g(z) = \phi_0 + \sum_j \phi_j z_j$ с ядровыми весами Шепли. Универсален, но дорог (экспоненциальное число коалиций).
TreeSHAP — model-specific метод, вычисляющий SHAP **точно и быстро** для деревьев и ансамблей (RF, GBDT) за полиномиальное время через динамическое программирование по путям дерева.

32.5 GradCAM

Grad-CAM (*Gradient-weighted Class Activation Mapping*) — метод локального объяснения для CNN, строящий тепловую карту важных областей изображения для класса c.
Пусть A^k — k-я карта признаков сверточного слоя ($H \times W$), y^c – скор класса c.
Алгоритм:

1. Веса каналов через градиенты: $\alpha_k^c = \frac{1}{HW}\sum_{i,j}\frac{\partial y^c}{\partial A_{ij}^k}$.
2. Линейная комбинация с ReLU: $L_{\mathrm{GradCAM}}^c(i,j) = \mathrm{ReLU}\left(\sum_k \alpha_k^c A_{ij}^k\right)$.
3. Апсемплинг карты L^c до размера входа.

Плюсы: универсален для CNN, наглядные карты "куда смотрит" модель. **Минусы:** не причинно зависит от слоя, грубое разрешение.

30 Градиентный бустинг (GB)

30.1 Процедура построения для MSE

Градиентный бустинг строит аддитивную модель вида: $F_M(x) = \sum_{m=0}^M \eta_m h_m(x)$, где h_m — базовые модели (неглубокие деревья), η — learning rate.
MSE: $\ell(y, F) = \frac{1}{2}(y - F)^2$. Инициализация: $F_0 = \bar{y}$.
Итерации:

1. Псевдо-остатки: $r_i^{(m)} = y_i - F_{m-1}(x_i)$.
2. *Обучение* h_m на остатках: $h_m \approx \arg\min_h \sum (r_i^{(m)} - h(x_i))^2$.
3. Подбор шага: $\gamma_m = \arg\min_\gamma \sum \frac{1}{2}(y_i - (F_{m-1}(x_i) + \gamma h_m(x_i)))^2$. Для деревьев: оптимальный вклад листа $c_{m,t} = \frac{1}{|R_{m,t}|}\sum_{x_i\in R_{m,t}} r_i^{(m)}$.
4. *Обновление:* $F_m = F_{m-1} + \eta\gamma_m h_m$.

30.2 Бустинг как градиентный спуск

Риск: $\mathcal{L}(F) = \sum \ell(y_i, F(x_i))$. Антиградиент: $r_i^{(m)} = -\frac{\partial \ell}{\partial F}|_{F=F_{m-1}}$. Аппроксимируем $h_m \approx \arg\min_h h(x_i))^2$.

30.2.1 вывод для кросс-энтропии

Бинарная классификация: $p = \sigma(F)$, log-loss $\ell(y, F) = -(y\log p + (1-y)\log(1-p))$.
Производная: $\frac{\partial \ell}{\partial F} = p - y$. Антиградиент: $\boxed{r_i^{(m)} = y_i - p_{m-1}(x_i)}$
Для log-loss: line search по γ_m или Newton step с весами $w_i = p_{m-1}(1 - p_{m-1})$.

30.3 Выбор базовой модели

Базовая модель — «слабый» алгоритм с ограниченной сложностью. На практике: неглубокие деревья (глубина 3–8), позволяющие моделировать взаимодействия признаков. Stumps (глубина 1) — простой baseline.
Критерии: скорость обучения/предсказания, контроль сложности (глубина, размеры листьев), согласованность с данными.

30.4 Параллелизм

GB последовательно по итерациям: $F_m = F_{m-1} + \eta h_m$. Параллелизм возможен внутри каждого шага и при инференсе.
Обучение: параллельный расчёт градиентов/остатков по объектам; параллельный поиск разбиений (feature-parallel/data-parallel); гистограммные методы; GPU-ускорение.
Инференс: $F_M(x) = F_0 + \eta\sum h_m(x)$ легко батчится и параллелится.

31 Библиотеки градиентного бустинга

31.1 XGBoost, LightGBM, CatBoost

GBDT — сильный подход для табличных данных. Основные промышленные библиотеки:
XGBoost: зрелая реализация с распределёнными обучением, GPU, кроме-виде алгоритмами. Сильная документация. Задачи: регрессия, классификация, ранжирование.
LightGBM: максимальная скорость/память через гистограммы, leaf-wise рост, GOSS/EFB. Подходит для больших данных.
CatBoost: сильные дефолты, категориальные признаки из коробки, ordered boosting (борьба с prediction shift). Минимум preprocessing.

31.2 Плюсы и минусы каждой

XGBoost: + документация, надёжность, распределённость. - может уступать по скорости.
LightGBM: + высокая скорость, гистограммы, GOSS/EFB. - leaf-wise требует контроли переобу
CatBoost: + удобство, категории из коробки, ordered boosting. - может быть тяжелее по памяти.
Выбор: много категорий → CatBoost; огромные данные → LightGBM; стабильность/документа → XGBoost.

33 Оптимизация гиперпараметров (HP)

33.1 Классы задач оптимизации

HPO: $\lambda^* \in \arg\min_{\lambda\in\Lambda} \mathcal{J}(\lambda)$, где $\mathcal{J}(\lambda)$ – качество на валидации. 0-order (black-box) оптимизация: нет градиентов.
Классы задач: непрерывные/дискретные/категориальные/смешанные пространства; tree-structured (условные параметры); невыпуклая, мультимодальная, шумная, дорогая $\mathcal{J}$; одно- или многокритериальная; multi-fidelity (оценка на меньшем ресурсе).

33.2 Основные подходы к оптимизации HP

Grid search: полный перебор по сетке. Плюсы: простота. Минусы: проклятие размерности.
Random search: сэмплирование $\lambda \sim p(\lambda)$. Плюсы: лучше использует бюджет. Минусы: не использует историю.
Байесовская оптимизация: суррогатная модель $\hat{\mathcal{J}}(\lambda)$ + acquisition function (EI, UCB).
TPE: оценивает $p(\lambda|\mathcal{J}$ мал) vs $p(\lambda|\mathcal{J}$ велик).
Эволюционные: популяция + мутации/кроссовер/отбор.
Multi-fidelity: Successive Halving, Hyperband — отсеи плохих конфигураций рано.

33.3 Библиотеки для оптимизации HP

Optuna: современная, TPE, pruners, удобный API. **Hyperopt:** классическая, TPE. **sklearn:** GridSearchCV, RandomizedSearchCV. **Ray Tune:** распределённые эксперименты. **SMAC, Nevergrad:** SMBO/эволюционные методы.

34 Матричное исчисление и производные матричных операций

34.1 Производная скалярного произведения

34.1.1 Для двух векторов

Далее считаем, что все векторы — **столбцы** ($x, y \in \mathbb{R}^d$), а градиент по вектору определён как

$$\nabla_x f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \cdots \\ \frac{\partial f}{\partial x_d} \end{pmatrix} \in \mathbb{R}^d.$$

Пусть

$$f(x, y) = x^\top y = \sum_{i=1}^d x_i y_i.$$

Тогда по компонентам:

$$\frac{\partial f}{\partial x_i} = y_i, \qquad \frac{\partial f}{\partial y_i} = x_i,$$

и, следовательно,

$$\boxed{\nabla_x(x^\top y) = y, \qquad \nabla_y(x^\top y) = x.}$$

Короткий вывод через дифференциалы (удобно для матричного исчисления).

$$df = d(x^\top y) = (dx)^\top y + x^\top (dy).$$

Сопоставляя с формой $df = (\nabla_x f)^\top dx + (\nabla_y f)^\top dy$, получаем:

$$(\nabla_x f)^\top dx = (dx)^\top y \Rightarrow \nabla_x f = y, \qquad (\nabla_y f)^\top dy = x^\top dy \Rightarrow \nabla_y f = x.$$

34.1.2 Для матрицы и вектора

На практике скалярное произведение часто возникает как произведение вектора на линейное преобразование. Рассмотрим матрицу $A \in \mathbb{R}^{m\times d}$ и векторы $x \in \mathbb{R}^d$, $b \in \mathbb{R}^m$. Определим скаляр:

$$f(x) = \langle Ax,\, b\rangle = (Ax)^\top b = b^\top Ax.$$

Это скалярное произведение двух векторов Ax и b.
Производная по x (матрица фиксирована). Перепишем:

$$f(x) = b^\top Ax = (A^\top b)^\top x.$$

Тогда по правилу из предыдущего пункта:

$$\boxed{\nabla_x\langle b^\top Ax\rangle = A^\top b.}$$

Альтернативный вывод через дифференциалы.

$$df = d(b^\top Ax) = b^\top A\,dx = (A^\top b)^\top dx \quad \Rightarrow \quad \nabla_x f = A^\top b.$$

Полезное обобщение (часто используется). Если

$$f(x, y) = y^\top Ax,$$

то

$$\boxed{\nabla_x(y^\top Ax) = A^\top y, \qquad \nabla_y(y^\top Ax) = Ax.}$$

(Это прямое следствие формы $df = y^\top A\,dx + (Ax)^\top dy$.)

35 Концепция нейросети

35.1 Мотивация нейросетей

Нейросети — композиция простых преобразований для аппроксимации сложных зависимостей.
Мотивация:

- Линейные модели требуют feature engineering для нелинейных зависимостей.
- NN автоматически извлекают признаки из данных (ранние слои — простые, глубокие — абстрактные).
- End-to-end обучение: $x \to \hat{y}$ по одной функции потерь.

35.2 Задача XOR

XOR: $\mathrm{XOR}(x_1, x_2) = \mathbb{I}[x_1 \neq x_2]$. Классы $\{(0,1),(1,0)\}$ и $\{(0,0),(1,1)\}$ линейно неразделимы. 1-слойный перцептрон не решает XOR. 2-слойная NN решает: скрытый слой создаёт нелинейные признаки (OR, AND), выходной их комбинирует.

35.3 Определения слоёв

35.3.1 Графическая нотация

Нейрон: $z = w^\top x + b$, $a = \sigma(z)$.
Слой: преобразование $x \mapsto h(x)$. Линейный слой: $h = Wx + b$, $W \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$. Активационный: $h = \sigma(z)$ (поэлементно).
Нотации количества слоёв: (1) только обучаемые (trainable), (2) все модули (Linear+Activatic (3) векторы активаций (input /hidden/ output).

35.4 Определение эмбеддинга

Эмбеддинг: векторное представление entity : $\mathcal{X} \to \mathbb{R}^d$ (слова, изображения, пользователи).
Кодирует «смысл» для измерения близости. Часто — выход предпоследнего слоя: $x \xrightarrow{\text{backbone}} e(x) \xrightarrow{\text{head}} \hat{y}$.
Основная идея backprop на вычислительном графе. Пусть в графе есть промежуточные величины $v_1, \dots, v_T$ и итоговый лосс L. Вводят **сопряжённые (adjoint) величины**:

$$\bar{v} \stackrel{\text{def}}{=} \frac{\partial L}{\partial v}.$$

Тогда для узла $v = h(u_1, \dots, u_k)$ локально применяют цепочку:

$$\bar{u}_j \mathrel{+}= \bar{v}\frac{\partial v}{\partial u_j}, \qquad j = 1, \dots, k,$$

и градиенты $\bar{\ }$ **в обратном порядке** (от L к входам/параметрам).
Часто используемые локальные производные (минимальный набор):

- $v = u + w \Rightarrow \frac{\partial v}{\partial u} = 1$, $\frac{\partial v}{\partial w} = 1$.
- $v = u \cdot w \Rightarrow \frac{\partial v}{\partial u} = w$, $\frac{\partial v}{\partial w} = u$.
- $z = Wx + b \Rightarrow \nabla_x L = W^\top \nabla_z L$, $\nabla_W L = (\nabla_z L)\,x^\top$, $\nabla_b L = \nabla_z L$.
- $\sigma(z) = \frac{1}{1+e^{-z}} \Rightarrow \sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- $\mathrm{ReLU}(z) = \max(0, z) \Rightarrow \mathrm{ReLU}'(z) = \mathbb{I}[z > 0]$ (субградиент в $z = 0$).

38.2 Посчитать простой пример вручную

Посчитаем backprop вручную на **простой сети с одним скрытым нейроном** и MSE.
Модель (граф).

$$z_1 = w_1 x + b_1, \qquad a_1 = \sigma(z_1), \qquad z_2 = w_2 a_1 + b_2, \qquad L = \frac{1}{2}(z_2 - y)^2.$$

Здесь $\sigma(\cdot)$ – сигмоида.
Числа для примера.

$$x = 1, \quad y = 0, \quad w_1 = 2, \quad b_1 = -1, \quad w_2 = -3, \quad b_2 = 0.5.$$

Forward pass.

$$z_1 = 2\cdot 1 + (-1) = 1, \qquad a_1 = \sigma(1) = \frac{1}{1+e^{-1}} \approx 0.731058579.$$
$$z_2 = (-3)\cdot 0.731058579 + 0.5 \approx -1.693175736.$$
$$L = \frac{1}{2}(z_2 - y)^2 = \frac{1}{2}(-1.693175736)^2.$$

(Значение L для градиентов не обязательно, важнее производные.)
Backward pass. Начинаем с

$$\bar{z}_2 = \frac{\partial L}{\partial z_2} = z_2 - y \approx -1.693175736.$$

36 Линейный слой (Linear layer)

36.1 Все названия линейного слоя

Линейный слой в нейросетях встречается под несколькими (почти эквивалентными) названиями:

- **Linear layer** (линейный слой) — наиболее частое название в PyTorch (`torch.nn.Linear`).
- **Dense layer** (плотный слой) — распространено в Keras/TensorFlow (`Dense`).
- **Fully Connected layer (FC)** (полносвязный слой) — подчёркивает, что каждый выход зависит от всех входов.
- **Affine layer** (аффинный слой) — более точное математическое название, т.к. обычно есть смещение b (строго линейным является Wx, а $Wx + b$ — аффинное отображение).
- **Perceptron layer** (слой перцептронов) — исторически/интуитивное название (нейроны = линейная комбинация + нелинейность).
- **Matrix multiplication + bias** (умножение на матрицу + bias) — инженерное описание операции.

Важно: в большинстве фреймворков под "линейным слоем" подразумевают именно **аффинное** преобразование $Wx + b$.

36.2 Формула

Пусть входной вектор $x \in \mathbb{R}^{d_{\text{in}}}$, выход $y \in \mathbb{R}^{d_{\text{out}}}$. Линейный (аффинный) слой задаётся:

$$y = Wx + b, \qquad W \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}},\ b \in \mathbb{R}^{d_{\text{out}}}.$$

Для батча $X \in \mathbb{R}^{n\times d_{\text{in}}}$ (каждая строка — объект) часто пишут:

$$Y = XW^\top + \mathbf{1}b^\top, \qquad Y \in \mathbb{R}^{n\times d_{\text{out}}},$$

где $\mathbf{1} \in \mathbb{R}^n$ — вектор из единиц (технически, это broadcast смещения).

36.3 Гиперпараметры, число параметров

Гиперпараметры линейного слоя (задаются до обучения):

- **Размер входа** d_{in} (`in_features`).
- **Размер выхода** d_{out} (`out_features`).
- **Наличие смещения** (`bias=True/False`).

Число обучаемых параметров:

- матрица весов W: $d_{\text{out}} \cdot d_{\text{in}}$ параметров;
- смещение b: d_{out} параметров (если bias включён).

Итого:

$$\#\text{params} = d_{\text{out}}d_{\text{in}} + \mathbb{I}[\text{bias}]\cdot d_{\text{out}}.$$

Пример. Если $d_{\text{in}} = 128$, $d_{\text{out}} = 64$, bias включён, то

$$\#\text{params} = 64\cdot 128 + 64 = 8192 + 64 = 8256.$$

36.4 Инициализация

Зачем инициализировать аккуратно. Если веса слишком большие, активации и градиенты могут **взрываться**; если слишком маленькие — могут **затухать** (особенно в глубоких сетях). Кроме того, **нельзя** инициализировать все веса одинаково (например, нулями), иначе нейроны слоя будут симметричны и будут обучаться одинаково (не произойдёт "разрыва симметрии").
Общая идея современной инициализации: выбрать случайные веса с дисперсией, зависящие от размеров слоя, чтобы сохранять масштабы сигналов при прохождении вперёд и назад.
Обозначим:

$$\mathrm{fan_in} = d_{\text{in}}, \qquad \mathrm{fan_out} = d_{\text{out}}.$$

Xavier/Glorot initialization (часто для tanh / sigmoid и близких режимов):

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{\mathrm{fan_in} + \mathrm{fan_out}}\right) \quad \text{или} \quad W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{\mathrm{fan_in} + \mathrm{fan_out}}},\ \sqrt{\frac{6}{\mathrm{fan_in} + \mathrm{fan_out}}}\right].$$

He/Kaiming initialization (часто для ReLU/LeakyReLU):

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{\mathrm{fan_in}}\right) \quad \text{или} \quad W_{ij} \sim \mathcal{U}\left[-\sqrt{\frac{6}{\mathrm{fan_in}}},\ \sqrt{\frac{6}{\mathrm{fan_in}}}\right].$$

Интуиция: ReLU обнуляет примерно половину активаций, поэтому дисперсию весов увеличивают относительно Xavier.
Смещения (bias). [1]часто инициализируют нулями:

$$b = 0,$$

поскольку симметрию нарушает именно случайность весов W, а bias обычно не требует случайно инициализации.
Практическая ремарка. Во многих фреймворках (PyTorch, TF) `Linear/Dense` уже имеет адекватную инициализацию по умолчанию; при построении глубоких сетей важно согласовывать выбор инициализации с активацией.

36.5 Теорема универсальной аппроксимации

Теорема универсальной аппроксимации (в контексте нейросетей) утверждает, что **достаточно широкая** полносвязная сеть с **одним скрытым слоем** и подходящей нелинейностью способна аппроксимировать любую непрерывную функцию на компакте.
Одна из стандартных формулировок (схематично): пусть $K \subset \mathbb{R}^d$ – компакт, $f : K \to \mathbb{R}$ – непрерывная функция, $\sigma(\cdot)$ – нелинейная активация (например, sigmoid, tanh, ReLU; важно, чтобы она была не "вырожденной"). Тогда для любого $\varepsilon > 0$ существует число нейронов m и параметры $\{a_i\}_{i=1}^m$, $\{w_i\}_{i=1}^m$, $\{b_i\}_{i=1}^m$ такие, что функция

$$F(x) = \sum_{i=1}^m a_i\,\sigma(w_i^\top x + b_i)$$

удовлетворяет

$$\sup_{x\in K} |f(x) - F(x)| < \varepsilon.$$

Как это связано с линейным слоем. Полносвязная сеть строится как композиция линейных (аффинных) преобразований и нелинейностей:

$$x \xrightarrow{Wx+b} z \xrightarrow{\sigma(\cdot)} h \xrightarrow{Wh+b} \hat{y}.$$

Один линейный слой без нелинейности — это лишь аффинное отображение и оно **не** универсальн Универсальность появляется именно из-за **композиции** линейных слоёв с **нелинейными** активациями.
Важные оговорки (что теорема не обещает):

- Теорема говорит о **существовании** параметров, но не гарантирует, что оптимизация (SGD/Adam) их найдёт быстро.
- Не гарантирует **эффективности**: для некоторых функций ширина m может потребоваться очень большая.
- В практике глубина (несколько слоёв) часто даёт **более параметрически эффективные** представления, чем один очень широкий слой.

37 Функции активации

37.1 Примеры (минимум 4)

Активация $\sigma : \mathbb{R} \to \mathbb{R}$ — поэлементная нелинейность: $a = \sigma(z)$. Без нелинейностей сеть остаётся линейной.
Основные активации:

1. **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Диапазон: $(0, 1)$.
2. **tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, $\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$. Диапазон: $(-1, 1)$.
3. **ReLU:** $\mathrm{ReLU}(z) = \max(0, z)$, $\mathrm{ReLU}'(z) = \mathbb{I}[z > 0]$.
4. **Leaky ReLU:** $\mathrm{LeakyReLU}(z) = \max(\alpha z, z)$, $\alpha \in (0, 1)$.
5. **ELU:** $\mathrm{ELU}(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1), & z \leq 0 \end{cases}$.
6. **GELU:** $\mathrm{GELU}(z) = z\Phi(z)$ (популярна в трансформерах).
7. **Softmax:** $\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_r e^{z_r}}$ (многоклассовая классификация).

37.2 Основные свойства (сравнительный анализ)

Ключевые свойства: нелинейность, диапазон, насыщение, zero-centered, гладкость, стабильно градиентами, sparsity, вычислительная стоимость.
Сравнение:

- **Sigmoid:** насыщается → vanishing gradients; не zero-centered. Использование: выход бинарно классификатора.
- **tanh:** zero-centered, но насыщается. Использование: RNN скрытые состояния.
- **ReLU:** *нет* насыщения на $z > 0$; риск dead ReLU на $z < 0$. Использование: CNN/MLP.
- **Leaky ReLU/ELU:** уменьшают dead ReLU проблему.
- **GELU:** гладкая, хорошо для трансформеров.
- **Softmax:** для многоклассового выхода с кросс-энтропией.

Практика: скрытые слои — ReLU/GELU; бинарный выход — sigmoid; многоклассовый — softmax.

Градиенты верхнего линейного слоя:

$$\frac{\partial L}{\partial w_2} = z_2 \cdot \frac{\partial z_2}{\partial w_2} = z_2 \cdot a_1 \approx (-1.693175736) \cdot 0.731058579 \approx -1.237810648,$$

$$\frac{\partial L}{\partial b_2} = z_2 \cdot \frac{\partial z_2}{\partial b_2} = z_2 \approx -1.693175736.$$

Протаскиваем градиент к a_1:

$$\bar{a}_1 = \frac{\partial L}{\partial a_1} = z_2 \cdot \frac{\partial z_2}{\partial a_1} = z_2 \cdot w_2 \approx (-1.693175736) \cdot (-3) = 5.079527208.$$

Производная сигмоиды в точке z_1:

$$\sigma'(z_1) = a_1(1-a_1) \approx 0.731058579 \cdot 0.268941421 \approx 0.196611933.$$

Значит

$$\bar{z}_1 = \frac{\partial L}{\partial z_1} = \bar{a}_1 \cdot \frac{\partial a_1}{\partial z_1} = \bar{a}_1 \cdot \sigma'(z_1) \approx 5.079527208 \cdot 0.196611933 \approx 0.998695663.$$

Градиенты нижнего линейного слоя:

$$\frac{\partial L}{\partial w_1} = \bar{z}_1 \cdot \frac{\partial z_1}{\partial w_1} = \bar{z}_1 \cdot x \approx 0.998695663 \cdot 1 \approx 0.998695663,$$

$$\frac{\partial L}{\partial b_1} = \bar{z}_1 \cdot \frac{\partial z_1}{\partial b_1} = \bar{z}_1 \approx 0.998695663.$$

Итог (градиенты параметров):

$$\boxed{\frac{\partial L}{\partial w_2} \approx -1.237810648, \quad \frac{\partial L}{\partial b_2} \approx -1.693175736, \quad \frac{\partial L}{\partial w_1} \approx 0.998695663, \quad \frac{\partial L}{\partial b_1} \approx 0.998695663.}$$

Проверка здравым смыслом (интуитивно): так как $z_2 < 0$ и $y = 0$, градиент $\bar{z}_2 = z_2 - y < 0$ — модель "хочет" подвить z_2 ближе к нулю. Из $\frac{\partial L}{\partial b_2} \approx -1.69$ следует, что шаг градиентного спуска увеличит b_2 (так как $b_2 \leftarrow b_2 - \eta\,\partial L/\partial b_2$), что действительно увеличивает z_2.

39 Многоклассовая классификация в нейросетях

39.1 Softmax

39.1.1 Основные свойства

Для K классов нейросеть выдаёт логиты $z \in \mathbb{R}^K$. **Softmax:**

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

Свойства: $\sum_k p_k = 1$; shift-инвариантность: $\mathrm{softmax}(z) = \mathrm{softmax}(z + c\mathbf{1})$; $p_a/p_b = \exp(z_a - z_b)$.
Якобиан: $\frac{\partial p_k}{\partial z_i} = p_k(\delta_{ki} - p_i)$.

39.1.2 Численная стабильность

Вычитают $z_{\max} = \max_j z_j$: $p_k = \frac{e^{z_k - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}$.
Log-softmax: $\log p_k = (z_k - z_{\max}) - \log \sum_j e^{z_j - z_{\max}}$.

39.2 Потеря кросс-энтропии

39.2.1 Определения информационного содержания и энтропии

Информация: $I(A) = -\log p$. **Энтропия:** $H(q) = -\sum_k q_k \log q_k$ (мера неопределённости).

39.2.2 Мотивация и смысл кросс-энтропии

Кросс-энтропия: $H(q,p) = -\sum_k q_k \log p_k$ (ожидаемое кодирование под p, если данные из q).
В ML: для one-hot $q_k = \mathbb{I}[k = y]$ имеем $H(q,p) = -\log p_y$ (максимизация вероятности правильного класса).
Градиент: для $p = \mathrm{softmax}(z)$ и one-hot q: $\frac{\partial}{\partial z_k}(-\sum_j q_j \log p_j) = p_k - q_k$.

39.2.3 Perplexity

$\mathrm{PPL} = \exp(H(q,p))$ или $2^{H(q,p)}$ (эффективное число равновероятных вариантов). Меньше $\Rightarrow$ лучше.

39.2.4 Дивергенция Кульбака–Лейблера

$D_{\mathrm{KL}}(q\|p) = \sum_k q_k \log \frac{q_k}{p_k} \geq 0$. Связь: $H(q,p) = H(q) + D_{\mathrm{KL}}(q\|p)$. Минимизация кросс-энтропии $\equiv$ минимизация $D_{\mathrm{KL}}(q\|p)$.

40 Компьютерное железо для ML

40.1 Архитектура фон Неймана

Классическая модель: программа и данные в общей памяти, процессор выполняет fetch–decode–execute.
Компоненты: CPU (Control Unit, ALU, регистры), память, шины.
Узкое место: фон-неймановское бутылочное горлышко — пропускная способность и латентност доступа к памяти.

40.1.1 Принцип локальности данных

Локальность: временная (недавно использованное будет использовано снова) и пространствени (соседние адреса). **Иерархия памяти:** регистры/L1 (нс) $\to$ L2/L3 (десятки нс) $\to$ DRAM (сотни нс) $\to$ SSD/HDD (мкс–мс) $\to$ сеть (мс+). Вывод: организовывать вычисления блоками с переиспользованием данных.

40.1.2 Поток данных при обучении модели

Диск $\to$ Dataset $\to$ CPU-RAM (подготовка, batching) $\to$ Dataloader (префетчинг) $\to$ GPU (VRAM) $\to$ compute (forward, backward, оптимизация). Bottleneck: медленный dataloader $\to$ простой GPU; малый батч $\to$ недогруженность GPU; частые CPU$\leftrightarrow$GPU копирования.

40.2 Таксономия Флинна

Классификация архитектур: SISD (один поток инструкций и данных), SIMD (одна инструкция, много данных), MISD (редко), MIMD (много инструкций и данных).

40.2.1 Место CPU и GPU

CPU: MIMD (разные ядра, разные потоки) + элементы SIMD (векторные расширения).
GPU: SIMD/SIMT (тысячи потоков, одна kernel-функция над разными данными).

40.2.2 CPU vs GPU, основные различия

CPU: низкая задержка, сложный control-flow, мало мощных ядер, большие кэши, универсальные потоки (MIMD).
GPU: высокая пропускная способность, data-parallel, много простых блоков, VRAM с высокой пропускной способностью, эффективен для плотных тензорных операций.
ML-следствие: обучение нейросетей быстрее на GPU; подготовка данных и сложный препроцес — на CPU.

40.3 NVIDIA CUDA, что это

CUDA (Compute Unified Device Architecture) — проприетарная (vendor-specific) платформа параллельных вычислений и API, которая позволяет использовать GPU NVIDIA для **ускорени вычислений общего назначения** (GPGPU).
Что даёт CUDA на практике:

- модель программирования, где разработчик описывает **kernel** (функцию), выполняемую параллельно на множестве потоков;
- управление **параллелизмом** (threads, blocks, grid) и **памятью** (копирование CPU↔GPU, размещение данных в VRAM);
- доступ к оптимизированным библиотекам для линейной алгебры и DL-операций (типовой стек: матричные умножения, свёртки и т.п.).

Связь с языком C/C++ (идея из лекций): CUDA API рассматривают как расширение, позволяющее задавать **параллелизм на уровне потоков** и использовать GPU-специфичные операции (включая передачу данных между CPU и GPU и использование виртуального "инстру слоя GPU).
Связь с Python/ML-фреймворками:

- NumPy ориентирован на векторизацию на CPU;
- PyTorch/TF могут исполнять тензорные операции на GPU, используя CUDA как backend (когда выбран device **cuda**).

41 Библиотека PyTorch

41.1 Зачем нужна ещё одна матричная библиотека

PyTorch — платформа для обучения нейросетей end-to-end: тензоры, **autograd** (автоматическое дифференцирование), исполнение на CPU/GPU, готовые модули. В отличие от NumPy, PyTorch автоматически строит вычислительный граф и вычисляет градиенты (backprop). Динамич граф, Python-уровень, GPU-ускорение $\to$ практичное обучение моделей.

41.2 Доступные бекенды

Устройства: CPU (`device='cpu'`), CUDA GPU (`device='cuda'`), AMD GPU (ROCm), Apple Silicon MPS (`device='mps'`), TPU/XLA (`torch_xla`).
Математические бекенды: BLAS/векторные оптимизации (CPU), библиотеки для матмулов (CUDA).
Распределённое обучение: communication backends (`nccl`, `gloo`, `mpi`).

41.3 Архитектура, основные модули и классы

41.3.1 nn.Module, Sequential, Optimizers, Schedulers, Criterions

Слой архитектуры: `torch.Tensor` (данные, устройство, dtype, `requires_grad`), `torch.autogr` (вычислительный граф, `backward()`), `torch.nn` (`nn.Module`, слои, потери), `torch.optim` (оптими `torch.utils.data` (`Dataset`, `DataLoader`).
`nn.Module`: базовый класс для моделей/слоёв. Параметры (`nn.Parameter`), `forward`, режимы `train()`/`eval()`, `to(device)`, `state_dict()`.
`nn.Sequential`: контейнер для последовательного применения модулей.
Optimizers: `SGD`, `Adam` — обновление параметров по градиентам. Шаги: `zero_grad` $\to$ `forward` $\to$ `loss` $\to$ `backward` $\to$ `step`.
Schedulers: изменение learning rate (`StepLR`, `CosineAnnealingLR`, `ReduceLROnPlateau`).
Criterions: `CrossEntropyLoss`, `BCEWithLogitsLoss`, `MSELoss`, `L1Loss`.

41.4 Dataflow в PyTorch, классы и их назначение

Путь данных: Storage $\to$ `Dataset` $\to$ Transforms $\to$ `DataLoader` $\to$ Batch (RAM) $\to$ Batch (device) $\to$ Model.
Dataset: `__len__()`, `__getitem__(i)` (Map-style) или `__iter__()` (IterableDataset).
Transforms: препроцессинг внутри Dataset или отдельными функциями.
DataLoader: `batch_size`, `shuffle`, `num_workers` (параллельная загрузка), `collate_fn` (сборка батча, например, из списка объектов сделать один батч-тензор или батч словарей).
Sampler / BatchSampler (контроль порядка и батчинга): `Sampler` задаёт порядок индексов, `BatchSampler` группирует индексы в батчи.
Train/Val/Test splits (практический поток). Часто делают три выборки и три DataLoader:

- **train_loader**: `shuffle=True`, используются градиенты и обновление параметров.
- **val_loader**: `shuffle=False`, измерение качества для подбора гиперпараметров/early stopping.
- **test_loader**: `shuffle=False`, финальная оценка.

Epoch и iteration (связь с SGD).

- **iteration (шаг)**: один батч $\to$ один шаг оптимизатора (`optimizer.step()`).
- **epoch (эпоха)**: один полный проход по train_loader (по всем батчам обучающей выборки).

Перенос батча на устройство и режимы вычислений.

- Обычно батч формируется в RAM, затем переносится на GPU: `x = x.to(device)`.
- Для инференса/валидации используют: `model.eval()` и `torch.no_grad()`, чтобы отключить dropout/обновление статистик нормировок и не хранить вычислительный граф (экономим память/время).

42 Обработка текстов

42.1 Подходы к токенизации (минимум 3)

Токенизация — разбиение исходного текста на последовательность единиц (токенов), с которой дальше работает модель:

$$\text{"сырой текст"} \longrightarrow (t_1, t_2, \ldots, t_n),$$

где токены t_i могут быть словами, подсловами, символами или специальными единицами. Токенизация определяет:

- размер словаря (vocabulary) и долю **неизвестных** токенов (OOV — out-of-vocabulary);
- длину последовательностей (а значит и стоимость обучения/инференса);
- устойчивость к опечаткам, редким словам, морфологии (особенно важно для русского языка).

Основные подходы к токенизации (минимум 3, на практике их больше):

1. **Токенизация по словам (word-level).**
   Самый простой подход: разделение по пробелам и знакам пунктуации (часто с регулярными выражениями).
   - **Плюсы:** интерпретируемо, короткие последовательности.
   - **Минусы:** большой словарь, проблема OOV, плохо переносит опечатки и богатую морфологию.
2. **Токенизация по символам (character-level).**
   Текст представляется как последовательность символов (включая пробелы/пунктуацию).
   - **Плюсы:** почти нет OOV (алфавит ограничен), высокая устойчивость к опечаткам.
   - **Минусы:** длинные последовательности $\Rightarrow$ дорогое обучение, сложнее захватывать дальние зависимости.
3. **Токенизация по подсловам (subword-level) — стандарт де-факто для трансформ**
   Идея: раскладывать слова на частотные части (морфемоподобные фрагменты), чтобы:
   - уменьшить OOV (редкие слова собираются из известных подслов);
   - ограничить размер словаря;
   - сохранить разумную длину последовательности.

   Примеры подсловых алгоритмов (их можно назвать как отдельные подходы):
   - **BPE** (Byte-Pair Encoding): итеративно склеивает наиболее частотные пары символов/п
   - **WordPiece**: близок к BPE, но выбор слияний основан на максимизации правдоподобия сегментации.
   - **Unigram LM** (SentencePiece Unigram): выбирает сегментацию вероятностной модельо подслов и оптимизирует словарь через "удаление" наименее полезных токенов.

   Часто используются специальные маркеры продолжения подслова (например, "##" или знак границы слова).
4. **N-граммная токенизация (обычно на уровне символов/слов).**
   Разбиение на последовательности фиксированной длины (bi-grams, tri-grams и т.д.).
   - **Плюсы:** может кодировать локальный контекст, устойчивее к редким словам, чем word-level.
   - **Минусы:** словарь и количество признаков быстро растут; часто используется в классич моделях (BoW/TF-IDF) и реже в современных DL-моделях как основная токенизация.
5. **Токенизация байтами (byte-level).**
   Представление текста как последовательности байтов (0–255) либо BPE поверх байтов.

- **Плюсы:** универсально для любых языков/символов, почти нет OOV.
- **Минусы:** более длинные последовательности, чем у subword; не всегда удобно интерпр

Практическая ремарка: для русского языка word-level токенизация сильно страдает из-за морфологии (много словоформ), поэтому подсловная токенизация (BPE/WordPiece/Unigram) обычно предпочтительнее.

42.2 Лемматизация, стемминг

Лемматизация — приведение слова к **лемме** (словарной форме) с учётом морфологии и контекста:

$$\text{"кошками"} \to \text{"кошка"}, \qquad \text{"went"} \to \text{"go"}.$$

- Обычно требует морфологического анализатора и словарей (или статистической модели).
- **Плюсы:** более "правильная" нормализация, полезна для поиска, классического NLP, некоторых задач тематического моделирования.
- **Минусы:** сложнее, медленнее, зависит от языка, возможны ошибки на омонимии и нестандартных формах.

Стемминг — эвристическое усечение слова до **основы (stem)** без гарантии получения реальной словарной формы:

$$\text{"connecting"} \to \text{"connect"}, \qquad \text{"машины"} \to \text{"машин"}.$$

- Как правило, это набор правил по удалению/замене суффиксов.
- **Плюсы:** быстро и просто, уменьшает словарь.
- **Минусы:** может давать несуществующие "корни", хуже различает смыслы, возможны ошибки двух типов:
  - **over-stemming:** разные слова склеиваются в один stem;
  - **under-stemming:** близкие формы остаются разными.

Когда что использовать (типовая логика):

- для классических пайплайнов (BoW/TF-IDF, простые классификаторы) стемминг/лемматиз часто улучшают качество;
- для современных нейросетей с subword токенизацией (BPE/WordPiece/Unigram) часто **не нужно** делать лемматизацию/стемминг, потому что модель и токенизатор уже обрабатывая вариативность форм.

42.2.1 Назвать базовые стеммеры

Классические (базовые) алгоритмы стемминга, которые обычно называют:

- **Porter Stemmer** (Алгоритм Портера) — один из самых известных для английского; правило-ориентированное удаление суффиксов.
- **Snowball Stemmer** (Porter2) — улучшенная и расширяемая версия, реализованная для многих языков.
- **Lancaster Stemmer** (Paice/Husk) — более агрессивный стеммер (сильнее "режет" слова).

Для русского языка часто упоминают:

- **Russian Snowball Stemmer** (вариант Snowball для русского);
- (на практике) **стеммеры на правилах суффиксов** в составе NLP-библиотек, аналогичны идеям Snowball.

43 Векторизация слов

Векторизация слов: $w \in \mathcal{V} \to x(w) \in \mathbb{R}^d$.

43.1 One-hot encoding

One-hot: $e(w) \in \{0,1\}^V$, где один компонент $= 1$, остальные $= 0$.
Недостатки: высокая размерность ($V \sim 10^5$–10^7), все слова ортогональны ($e(w_1)^\top e(w_2) = 0$) — нет семантики, плохо с OOV/морфологией. **Вывод:** нужны плотные эмбеддинги.

43.2 Word2Vec, GloVe, FastText

43.2.1 Постановка задачи, функция потерь, признаки

Distributional hypothesis: слова в похожих контекстах $\to$ близкие векторы.
Word2Vec: CBOW (контекст$\to$слово) или Skip-gram (слово$\to$контекст). Skip-gram: $p(w_o|w_i) = \frac{\exp(v'^\top_{w_o} v_{w_i})}{\sum_w \exp(v'^\top_w v_{w_i})}$. Потери: $-\sum_t \sum_{j \neq 0} \log p(w_{t+j}|w_t)$. Negative sampling: бинарная классификация вместо полного softmax.
GloVe: матрица X_{ij} — совместная встречаемость. Цель: $v_i^\top u_j + b_i + c_j \approx \log X_{ij}$. Потери: $\sum_{i,j} f(X_{ij})(v_i^\top u_j + b_i + c_j - \log X_{ij})^2$.
FastText: слово = сумма n-грамм: $v_w = \sum_{g \in G(w)} z_g$. Плюсы: морфология, OOV.
Сравнение: Word2Vec — предсказательная (локальная), GloVe — count-based (глобальная), FastText — Word2Vec + subword.

44 Векторизация текста

44.1 Bag-of-words

BoW — документ как «мешок слов»: $x(d) = (x_1, \ldots, x_V) \in \mathbb{R}^V$, где $x_j = \#(w_j \in d)$ (count) или $\mathbb{I}[w_j \in d]$ (binary).
Плюсы: простота, скорость, интерпретируемость. **Минусы:** теряет порядок слов, высокая размерность/разреженность, не учитывает синонимию.

44.2 TF-IDF

TF-IDF: взвешивание BoW. tfidf$(t,d) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t)$, где $\mathrm{idf}(t) = \log \frac{N+1}{\mathrm{df}(t)+1} + 1$.
Варианты TF: $\#(t \in d)$, $\frac{\#(t,d)}{|d|}$, $1 + \log(\#(t \in d))$.
Смысл: частые в документе, но редкие в коллекции слова $\to$ высокий вес. **Плюсы:** простота, прирост качества. **Минусы:** игнорирует порядок/семантику, требует корпус.

44.3 * BM25

BM25 (IR-ранжирование, развитие TF-IDF):

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{idf}(t) \cdot \frac{\mathrm{tf}(t,d)(k_1+1)}{\mathrm{tf}(t,d) + k_1(1 - b + b \cdot |d|/\mathrm{avgdl})}.$$

Учитывает насыщение TF и нормировку по длине документа. Параметры: $k_1 \approx 1$-2, $b \approx 0.75$.
Использование: текстовый поиск, RAG sparse retrievet, гибридный поиск. Обычно лучше TF-IDF в IR-задачах.

45 Экспоненциальное сглаживание

45.1 Формула

Экспоненциальное сглаживание (simple exponential smoothing, SES; также EWMA — exponentially weighted moving average) используется для сглаживания временного ряда y_t и получения сглаженной оценки уровня s_t.
Рекуррентная формула (основная):

$$\boxed{s_t = \alpha\, y_t + (1-\alpha)\, s_{t-1}, \qquad 0 < \alpha \leq 1.}$$

где:

- y_t — наблюдение ряда в момент t,
- s_t — сглаженное значение (оценка уровня) в момент t,
- α — коэффициент сглаживания (чем больше α, тем сильнее учитывается текущее наблюден

Прогноз на один шаг вперёд (типичная интерпретация SES):

$$\boxed{\hat{y}_{t+1|t} = s_t.}$$

Эквивалентная форма для прогноза: если обозначить текущий прогноз как $\hat{y}_{t|t-1}$, то

$$\boxed{\hat{y}_{t+1|t} = \alpha\, y_t + (1-\alpha)\, \hat{y}_{t|t-1}.}$$

Развёрнутая форма (показывает веса прошлых наблюдений): раскрывая рекурсию, получаем

$$s_t = \alpha y_t + \alpha(1-\alpha) y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \cdots$$

или компактно:

$$\boxed{s_t = \alpha \sum_{k=0}^{\infty} (1-\alpha)^k\, y_{t-k} \quad \text{(при достаточно "давнем" старте).}}$$

При этом веса неотрицательны и (в пределе) суммируются к 1:

$$\alpha \sum_{k=0}^{\infty} (1-\alpha)^k = 1.$$

Инициализация. Нужно задать s_0 (например, $s_0 = y_0$ или среднее первых наблюдений). На коротких рядах выбор s_0 может заметно влиять на начало сглаженной кривой.

45.2 Мотивация названия

Название "экспоненциальное сглаживание" отражает два факта:
1) "Экспоненциальное" — из-за формы весов.
Во взвешенной сумме веса прошлых наблюдений убывают **экспоненциально** по мере удаления во времени:

$$w_k = \alpha(1-\alpha)^k, \qquad k = 0, 1, 2, \ldots$$

То есть наблюдение y_{t-k} влияет тем меньше, чем больше лаг k, и убывание происходит геометрически (экспоненциально по k).
2) "Сглаживание" — потому что подавляется шум.
Операция

$$s_t = \alpha y_t + (1-\alpha) s_{t-1}$$

является фильтрацией: случайные высокочастотные колебания (шум) в y_t усредняются, и s_t меняется более плавно. Параметр α задаёт компромисс:

- большое $\alpha \Rightarrow$ меньше сглаживание, быстрее реакция на изменения;
- малое $\alpha \Rightarrow$ сильнее сглаживание, но медленнее реакция на сдвиги уровня.

46 Индуктивные смещения (inductive bias) в Deep Learning

46.1 Регуляризация

Индуктивное смещение — предположения, встроенные в модель/алгоритм для обобщения.
Регуляризация: минимизация $\frac{1}{N}\sum_i \ell(y_i, f(x_i;\theta)) + \lambda\Omega(\theta)$, где $\Omega(\theta)$ — штраф.
Типы: L2 ($\|\theta\|_2^2$, weight decay), L1 ($\|\theta\|_1$, разреженность), early stopping, аугментация, dropout, label smoothing.

46.2 Batch normalization, layer normalization

Batch Normalization (BN): нормирует по батчу: $\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$, $y = \gamma\hat{x} + \beta$. Плюсы: стабилизация высокие LR. Минусы: чувствительна к batch size.
Layer Normalization (LN): нормирует по признакам внутри объекта (не зависит от batch size). Популярна в трансформерах.

46.3 Dropout

Dropout: случайно выключает нейроны при обучении. Inverted dropout: $\tilde{h}_i = \frac{m_i}{1-p} h_i$, где $m_i \sim \mathrm{Bernoulli}(1-p)$.
Режимы: обучение — dropout включён (регуляризация); инференс — выключен (все нейроны активны).

46.4 Аугментации

Аугментации: преобразования данных $\tilde{x} = T(x)$, сохраняющие метку. Вносят инвариантность к трансформациям.
Примеры: CV (crop, flip, rotation, cutout, mixup), NLP (dropout токенов, синонимы), аудио (time shift, noise).
Библиотеки: torchvision.transforms, Albumentations, imgaug, kornia.
TTA (Test-time augmentations): применение аугментаций на инференсе с усреднением предсказаний. Плюсы: выше качество. Минусы: медленнее в M раз.

47 Методы оптимизации

47.1 Gradient descent, SGD, Momentum, RMSProp, Adam

Обозначения: θ — параметры, $L(\theta)$ — потери, $g_t = \nabla_\theta L(\theta_t)$, η — learning rate.
GD: $\theta_{t+1} = \theta_t - \eta g_t$ (градиент по всей выборке; стабильно, но дорого).
SGD: градиент по мини-батчу B_t: $\theta_{t+1} = \theta_t - \eta g_t$ (быстро, но шумно).
Momentum: сглаживание: $v_t = \mu v_{t-1} + g_t$, $\theta_{t+1} = \theta_t - \eta v_t$, $\mu \in [0,1)$.
RMSProp: адаптивный шаг: $s_t = \rho s_{t-1} + (1-\rho) g_t^2$, $\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_{t+1}}}$.
Adam: momentum + RMSProp: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$, bias-correction $\hat{m}_t, \hat{v}_t$, $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$.

47.2 Muon

Muon (MomentUm Orthogonalized by Newton–Schulz) — оптимизатор для 2D-матриц весов: $M_t = \mu M_{t-1} + G_t$ (momentum), затем $\hat{M}_t = \mathrm{NewtonSchulz}(M_t)$ (ортогонализация), $W_{t+1} = W_t - \eta \hat{M}_t$.
Смысл: ортогонализация делает шаг более изотропным. Применяют к скрытым слоям, bias/эмбе — обычным оптимизатором.

47.3 Расход памяти для каждого

Память оптимизатора (d — параметров):

Метод	Доп. состояние
GD/SGD	$0 \cdot d$
Momentum	$1 \cdot d$ (буфер v)
RMSProp	$1 \cdot d$ (буфер s)
Adam	$2 \cdot d$ (m, v)
Muon	$\approx 1 \cdot d$ (momentum)

В FP32: Adam требует $\approx 8d$ байт доп. памяти.

48 Ранжирование

48.1 Постановка задачи

Постановка задачи (learning-to-rank).
Дано N запросов $Q = \{q_1, \ldots, q_N\}$. Для каждого q_i есть набор документов $D_i = \{d_{i1}, \ldots, d_{iK_i}\}$ и метки релевантности $L_i = \{l_{i1}, \ldots, l_{iK_i}\}$, где $l_{ij} \in \mathbb{R}$.
Цель: построить модель $f(q_i, d_{ij}) = s_{ij}$, чтобы сортировка по s_{ij} согласовывалась с сортировкой по l_{ij}:

$$\mathrm{argsort}(s_{i1}, \ldots, s_{iK_i}) \approx \mathrm{argsort}(l_{i1}, \ldots, l_{iK_i}).$$

Важно: требуется восстановить **порядок**, а не точные значения l_{ij}.

48.1.1 Отличия от регрессии и классификации

- **Групповая структура:** данные сгруппированы по запросам, качество оценивается внутри групп.
- **Оптимизация порядка:** важны относительные сравнения (кто выше), особенно в топе.
- **Градуированные метки:** l_{ij} — уровни релевантности (0–4), не только бинарные.
- **Другие loss'ы:** pairwise (RankNet), listwise (ListNet), метрики (mAP, NDCG, MRR).
- **Цена зависит от позиции:** ошибка на позициях 1-2 хуже, чем на 97-98.

48.2 Оффлайн-метрики

48.2.1 mAP

Average Precision@K (AP@K).
Для запроса q с упорядоченным списком K документов: $P@k = \frac{1}{k}\sum_{i=1}^{k} \mathrm{rel}(i)$ — precision на первых k, где $\mathrm{rel}(k) \in \{0,1\}$ — индикатор релевантности.

$$\boxed{AP@K = \frac{1}{\min(R,K)} \sum_{k=1}^{K} P@k \cdot \mathrm{rel}(k).}$$

где R — число релевантных документов в разметке.
mAP@K — среднее AP@K по запросам: $\boxed{mAP@K = \frac{1}{N}\sum_{i=1}^{N} AP_i@K.}$
Смысл: награждает ранжирование, где релевантные документы встречаются раньше. Ограничи бинарная релевантность, косвенный учёт позиций.

48.2.2 NDCG

DCG@K: $\boxed{DCG@K = \sum_{k=1}^{K} \frac{2^{l_k} - 1}{\log_2(k+1)}}$, где l_k — релевантность документа на позиции k. Числит усиливает разницу уровней, знаменатель — логарифмическая скидка по позиции.
NDCG@K: нормировка на идеальное DCG: $\boxed{NDCG@K = \frac{DCG@K}{IDCG@K} \in [0,1].}$ Поддерживает градуированные метки и акцент на верх выдачи.

48.2.3 MRR

Reciprocal Rank: $RR_i = \frac{1}{\mathrm{rank}_i}$, где rank_i — позиция первого релевантного документа.
MRR: $\boxed{MRR = \frac{1}{N}\sum_{i=1}^{N} \frac{1}{\mathrm{rank}_i}}$ Полезна, когда важно, чтобы хотя бы один релевантный был как можно выше.

48.3 Онлайн-метрики

48.3.1 Как измеряются

Онлайн-метрики измеряются на реальных пользователях через **A/B-тесты**: пользователей распределяют по вариантам (control vs treatment), фиксируют события и сравнивают метрики между группами.
Практический приём: агрегировать на уровне пользователя: $\hat{m}_{\text{юзер}} = \frac{1}{U}\sum_{u=1}^{U} m(u)$, а не сырую сумму событий (снижает влияние роботов/спама).
Почему онлайн — "истина": учитывает динамику продукта, новых пользователей, UX-эффекты и реальную экономику.

48.3.2 Дерево метрик, North star метрика

Дерево метрик: иерархия метрик от технических (CTR, конверсия) до агрегированных (retention, GMV) и главной на вершине.
North Star metric: главная метрика продукта, которая максимально близка к долгосрочной ценности, согласует команды и разрешает конфликты метрик.

48.4 Функции потерь RankNet

48.4.1 Что она предсказывает

RankNet — **pairwise** подход. Для пары документов (i,j) модель выдаёт скоры s_i, s_j и предсказы вероятность $i \succ j$:

$$\boxed{P_{ij} = \mathbb{P}(i \succ j) = \frac{1}{1 + \exp(-\sigma_0(s_i - s_j))}.}$$

где $\sigma_0 > 0$ — масштаб. Целевая метка пары: $S_{ij} = 1$ если $l_i > l_j$, -1 если $l_i < l_j$, 0 если равны. Преобразуем: $P_{ij}^* = \frac{1+S_{ij}}{2}$.
Функция потерь (кросс-энтропия):

$$\boxed{\mathcal{L}_{ij} = -P_{ij}^* \log P_{ij} - (1 - P_{ij}^*) \log(1 - P_{ij}).}$$

Компактная форма: $\boxed{\mathcal{L}_{ij} = \log(1 + \exp(-\sigma_0 S_{ij}(s_i - s_j))).}$

48.4.2 * Lambda-gradients

Градиенты RankNet по скорам: $\boxed{\frac{\partial \mathcal{L}_{ij}}{\partial s_i} = \sigma_0(P_{ij} - P_{ij}^*), \quad \frac{\partial \mathcal{L}_{ij}}{\partial s_j} = -\sigma_0(P_{ij} - P_{ij}^*).}$
Обозначение: $\lambda_{ij} = \sigma_0(P_{ij} - P_{ij}^*)$. Суммарная lambda для документа: $\boxed{\lambda_i = \sum_j \lambda_{ij} - \sum_j \lambda_{ji}.}$
LambdaRank: взвешивает λ_{ij} на $|\Delta NDCG|$ при перестановке пары: $\lambda_{ij}^{\text{LambdaRank}} \propto \lambda_{ij}^{\text{RankNet}} \cdot |\Delta \mathrm{NDCG}_{ij}|$, сильнее исправляя критичные ошибки в топе.

48.4.3 Другие функции потерь (просто назвать)

- **LambdaRank**
- **LambdaMART**
- **YetiRank**
- **ListNet**
- **ListMLE**

49 Uplift modelling

49.1 Постановка задачи, отличие look-alike и response моделей

Uplift modelling — оценка **прироста** отклика от воздействия vs отсутствия воздействия.
Постановка: X — признаки, $T \in \{0,1\}$ — воздействие, Y — исход. Потенциальные исходы: $Y(1), Y(0)$. Наблюдаем $Y = TY(1) + (1-T)Y(0)$.
CATE (uplift): $u(x) = \mathbb{E}[Y \mid X = x, T = 1] - \mathbb{E}[Y \mid X = x, T = 0]$.
Цель: ранжировать по $u(x)$, максимизируя инкрементальную прибыль.
Отличия:

- **Response models** предсказывает $\mathbb{P}(Y = 1 \mid X, T = 1) \to$ выбирает «sure things» (купят без воздействия).
- **Look-alike** ищет похожих на ценных пользователей $\to$ не отвечает на причинный вопрос.

Сегменты: Sure things ($Y(1) = Y(0) = 1$), Lost causes ($Y(1) = Y(0) = 0$), **Persuadables** ($Y(1) = 1, Y(0) = 0$, цель!), Do-not-disturb ($Y(1) = 0, Y(0) = 1$, избегать!).

49.2 Two-model uplift technique

Two-model (T-learner): обучаем две модели: $p_1(x) \approx \mathbb{P}(Y = 1 \mid X = x, T = 1)$, $p_0(x) \approx \mathbb{P}(Y = 1 \mid X = x, T = 0)$. Uplift: $\hat{u}(x) = \hat{p}_1(x) - \hat{p}_0(x)$.
Плюсы: простота, любые модели. **Минусы:** ошибки накапливаются в разности; при нерандомиз нужна корректировка (IPW/doubly robust).

49.3 Class transformation technique

Преобразуем uplift в классификацию. Новая метка: $Z = 1$ если ($T = 1 \wedge Y = 1$) или ($T = 0 \wedge Y = 0$), иначе $Z = 0$ (т.е. $Z = \mathbb{I}[Y = T]$).
При сбалансированной рандомизации ($\mathbb{P}(T = 1) = \frac{1}{2}$): $u(x) = 2\hat{p}_Z(x) - 1$, где $\hat{p}_Z(x) = \mathbb{P}(Z = 1 \mid X = x)$.
Плюсы: любой классификатор. **Минусы:** формула требует 50/50 раздачи.

49.4 Библиотеки для uplift modelling

Python: scikit-uplift, causalml, EconML, DoWhy. **R:** uplift, grf, causalTree.
На практике часто используют градиентный бустинг (CatBoost/LightGBM) с мета-алгоритмами (T/S/X-learner).

49.5 Примеры применения

Маркетинг/CRM (промо), удержание (churn), персонализация офферов, финтех (лимиты), здравоохранение (CATE терапии), продуктовые эксперименты.